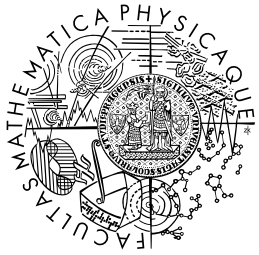

FACULTY
OF MATHEMATICS
AND PHYSICS
Charles University

BACHELOR THESIS

Adam Balko

Arduino platform simulator

Department of Software and Computer Science Education

Supervisor of the bachelor thesis: RNDr. David Bednárek, Ph.D.

Study programme: Computer Graphics, Vision and Computer Games Development

Prague 2026

I declare that I carried out this bachelor thesis on my own, and only with the cited sources, literature and other professional sources. I understand that my work relates to the rights and obligations under the Act No. 121/2000 Sb., the Copyright Act, as amended, in particular the fact that the Charles University has the right to conclude a license agreement on the use of this work as a school work pursuant to Section 60 subsection 1 of the Copyright Act.

In ..... date ....

Author's signature

I dedicate this work to my parents, who give me their full support with whatever endeavor I choose to pursue. My dedication also goes to my friends Marek and Andrej, who made these few years of study unforgettable.

Title: Arduino platform simulator

Author: Adam Balko

Department: Department of Software and Computer Science Education

Supervisor: RNDr. David Bednárek, Ph.D., Department of Software Engineering

Abstract: This thesis focuses on simulating code of Arduino microcomputers in a virtual environment with a graphical user interface without the need of emulation. An important part of the thesis shows how and why should be the simulator and the GUI split into two processes, and how to keep them synchronized. This introduces the reader into the field of interprocess communication and its implementations using TCP connection, shared memory and events. Another important part focuses on simulation of digital circuits with extensible component library. It describes how new components and circuits can be created by the user from YAML, SVG and C# files.

Keywords: digital circuit simulator, Arduino

Název práce: Simulátor prostředí Arduino

Autor: Adam Balko

Katedra: Katedra softwaru a výuky informatiky

Vedoucí bakalářské práce: RNDr. David Bednárek, Ph.D., Katedra softwarového inženýrství

Abstrakt: Tato práce se zaměřuje na simulaci kódu mikropočítačů Arduino ve virtuálním prostředí s grafickým uživatelským rozhraním bez nutnosti emulace. Důležitá část práce ukazuje, jak a proč by měly být simulátor a GUI rozděleny do dvou procesů a jak je udržovat synchronizované. To čtenáře uvádí do oblasti meziprocesové komunikace a její implementace pomocí TCP spojení, sdílené paměti a událostí. Další významná část se zaměřuje na simulaci digitálních obvodů s rozšiřitelnou knihovnou komponent. Popisuje, jak může uživatel vytvářet nové komponenty a obvody ze souborů YAML, SVG a C#.

Klíčová slova: simulátor číslicových obvodů, Arduino

Contents

Introduction

Arduino is a very popular microcomputer among amateurs and professionals alike. Despite this, testing the code for an Arduino machine can be fairly difficult. Debugging options are limited and the user needs to rely on additional measuring equipment. The usual choice is to test the code with an AVR emulator. Emulation is the process of running machine code on a hardware that doesn't match its instruction set. AVR is the processor that controls the Arduino board. These emulators are sometimes part of a bigger logical circuit simulation system, other times they are just a single piece of software.

Goal of this work is to skip the emulation process and create a working Arduino simulator by mimicking the changes of digital values on the board's pinout. The user code will be compiled directly to the x86-64 architecture instruction set, which is our target platform, and thus able to run at the native speed of the host machine.

Bundled with this simulator is a graphic application for creating and testing logical circuits. This GUI is written with Avalonia framework and will contain Arduino as it's main component. The point of this application is to attach to the running Arduino simulator and visualize the expected Arduino behavior with user's code on a given circuit. We want to give the user the power to compose their own circuits, and define the behavior and looks of their components. Additionally, the user will be able to debug their program, while the GUI is running, using their preferred IDE and C++ debugger, providing real-time feedback and inspection when stepping through the code.

Structure of the thesis

This thesis starts with a brief introduction of technologies we've chosen for achieving our vision from the introduction. Then we lay down very specific goals and requirements, we want to address in this very thesis, using the mentioned technologies. In chapter 1 we introduce the reader to programming for Arduino boards. We show what is required from Arduino programs (1.1), how they access the board's I/O (1.2), and how are they built (1.3).

In chapter 2 we provide a high level look on the core systems of the work. We show why it is required to split the work into 2 applications (or more accurately processes) and we expose the reader to the world of inter-process communication (2.2). We show how it can be achieved using TCP connection (2.2.4), shared memory (2.2.5) and events (2.2.3), and what kind of problems had to be taken into consideration (2.2.1). Then we delve into more details on how the individual projects and classes have been implemented in chapters 3 and 4.

Finally, the chapter 5 consists of specifics on how to simulate Arduino code using the applications we developed for this thesis (5.1). We specify how to create custom components and scenes for the circuit simulator (5.2). And in the very end, we specify what interprocess communication protocols we have created for interacting with the simulator's backend, so that any other user interface can potentially be connected to it (5.3).

0.1 Technologies used

C++, CMake, Boost

For technical reasons described in the subsection 2.2.1, the work is split into two processes: the frontend and the backend. The backend is written in C++, as it is almost identical to the language used for Arduino sketches (see 1.3). This makes the build process of the user's code much simpler. In the C++ project we use Boost libraries that provide useful abstractions for communicating with other processes (`asio`, `interprocess`) and parsing command line arguments (`program_options`). We use CMake as our metabuild system of choice, as it is the one we are the most familiar with.

C#

The frontend is written in C# from the .NET 9 Software development kit. We consider the language to be more convenient for writing many of the essential systems than its C++ counterpart. That includes handling the graphics, the user interactions, reflection and parsing numerous configuration files. We also believe the language will be more comfortable for users that wish to extend the simulator with their own components. Libraries are also much easier to set up and compile thanks to NuGet.

Avalonia UI

For the GUI we decided to use Avalonia UI. Avalonia is a popular open-source cross-platform .NET framework, used by major companies like JetBrains or Unity. It is based on the Microsoft's WPF framework utilizing the same XAML markup language and many of the same core principles. We chose it specifically because neither WPF or its cross-platform successor MAUI are available on Linux, one of our target platforms.

SVG

We want to render the circuit's components using vector graphics, as they don't have a limited resolution. This is crucial, since we also want to render the components in different sizes and under different zoom levels. The most common file format for vector graphics is SVG, so this is the format we will require to be supplied for custom components. Avalonia provides a sufficient support for rendering vector graphics using the Skia engine under the hood, as all of its built-in controls are rendered with it.

0.2 Specific goals of the thesis

Target platforms:

- Linux (x86-64)
- Windows 10 or higher (64-bit)

Software prerequisites:

- arduino-cli [1]
- Minimal system requirements of Avalonia UI [2]
- .NET 9 SDK or higher
- C++ compiler
- CMake
- C++ Boost libraries:
  - asio
  - interprocess
  - program_options

Simulator functional requirements

- Defines platform independent constants and macros from `Arduino.h`.
- Implements `digitalWrite`, `digitalRead`, `pinMode` and `millis` functions from `Arduino.h`.
- Declares and calls `setup` and `loop` functions with the behavior expected on an Arduino machine. [3]
- Compiles and runs any valid `Arduino UNO R3 SMD` sketch, preprocessed with the `arduino-cli` tool, containing only symbols mentioned above.
- Reflects behavior of the above functions in an internal Arduino state. Logs changes of this state in a text file.
- Integrates an interprocess communication mechanism for exchanging data with an external GUI application.
- The events incoming from a GUI application must be resolved in a manner, that preserves the continuity of events.

GUI functional requirements

- The application must stay responsive during debugging of the simulator process.
- The application must be able to connect to the interprocess mechanism of the simulator.
- Simulates simplified digital circuits. Exact physical properties, like voltages and currents might not be implemented.

[1] See section 1.3

[2] See `https://docs.avaloniaui.net/docs/supported-platforms`

[3] The significance of the mentioned functions is described in detail in the chapter 1

- Implements mechanism for defining custom components and visuals with `.yaml`, `.cs` and `.svg` files. These components include interactive controls, such as buttons.
- Implements mechanism for defining a circuit (scene) using the custom components.
- Predefines LED, button, voltage source and Arduino components, and a scene which utilizes these components.
- The changes to the simulator's internal state must be immediately reflected on visual components connected to the Arduino in the scene.
- The scene is able to be zoomed and panned. The visuals of the components must be rendered using vector graphics.
- Multiple interactive components may be pressed at once.

1 Introduction to Arduino programming

1.1 Sketches

The Arduino programs are commonly referred to as *sketches*. A sketch is not a single file, but an entire directory, usually populated by files with the `.ino` extension, containing the source code of the program. `.cpp`, `.c`, `.S` (assembly files), and header files like `.h` are also accepted as files containing source code. Every sketch must contain a `.ino` file with a file name matching the sketch root directory name [1].

The Arduino programming language, in which `.ino` files are written, is a C++ dialect with very subtle differences, mostly concerning linking of files and omitting function declarations. Bodies of function definitions and variable declarations can be considered as a perfectly valid C/C++ code.

The essential functions that an Arduino programmer has to define in their sketch are `void setup()` and `void loop()`. The `setup()` function is called once, when the program is booted. That is when the Arduino board is powered up, the program is uploaded to it, or when the reset button is pressed. The `loop()` function is called in a loop indefinitely. This is the proper place to define any interactions with the board, detecting inputs, updating timers, etc.

1.2 Arduino board interface

The target board `Arduino Uno R3 SMD` provides 6 analog and 14 digital general purpose pins, some of which can be used for I2C and SPI protocol communication, or PWM modulation. The most simple to use and simulate are the digital pins for general purpose I/O, so this thesis focuses exclusively on using these pins as such. [2]

The digital pins can be manipulated using only these 3 functions:

`void pinMode(pin, mode)` configures specified pin either as an input or an output

`value digitalRead(pin)` reads the voltage on the pin, stores it as a digital value, either 1 or 0

`void digitalWrite(pin, value)` when set to output, sets the voltage of the pin to corresponding value.

Pins are identified by their integer index. `mode` and `value`'s underlying type is also an integer, but it is preferred to use predefined constants, such as `INPUT`/`OUTPUT`, `HIGH`/`LOW` and such. Below is an example of a sketch from the Arduino's documentation, using only the described functions:

Listing 1.1 Sketch example [3]

```
1 int ledPin = 13;  // LED connected to digital pin 13
```

```
int inPin = 7;     // pushbutton connected to digital pin 7
int val = LOW;     // variable to store the read value

void setup() {
  pinMode(ledPin, OUTPUT);  // sets the digital pin 13 as output
  pinMode(inPin, INPUT);    // sets the digital pin 7 as input
}

void loop() {
  val = digitalRead(inPin);   // read the input pin
  digitalWrite(ledPin, val);  // sets LED to the button's value
}
```

Another simple, but very useful function to have is `millis`, which returns the amount of milliseconds passed since the sketch has started to run. Without it, we couldn't implement things like clocks, timed outputs or button debouncing.

1.3 Build process

Before the code is compiled, all `.ino` go through a simple preprocessing stage. `#line` directives are added. These link to the original line and file, when the code is debugged. All files are concatenated together, function declarations are generated at the top, and the `Arduino.h` is included. This header file is crucial as it declares the `setup` and `loop` functions, functions for manipulating pins, common constants, mathematical functions, etc. At this point the sketch becomes a complete C++ source code.

It is also at this where we steer away from the standard compilation process, and apply our own `Arduino.h` implementation in our simulator. Usually, the C++ code would be linked to an implementation that contains binaries and constants for the given microchip installed on the target board. Then it would be compiled with a compiler for the given instruction set (e.g. `avr-gcc`). The code would be then transmitted over USB to the user's Arduino [4].

In our case, we include the entire preprocessed sketch in our simulator project written in C++, and compile it as whole for the instruction set of the host machine. This executable will implement mock functions declared in the `Arduino.h`, Arduino's internal state, and a way to communicate the change of state to an external GUI application.

The entire process from preprocessing to compilation, can be orchestrated with the use of `arduino-cli` command line application. We will use it to preprocess the `.ino` for us, so that we end up with a clean C++ file we can include in our project.

2 Architecture design

2.1 Architecture overview

The whole project is divided into two main parts. The purpose of the backend is to execute the Arduino code and simulate the change on the board's pinout. The purpose of the frontend is to reflect these changes on a user defined logical circuit, while offering the user control over the simulation. We chose to implement the backend in C++ and the frontend in C#. Both backend and frontend are running in its own separate process.

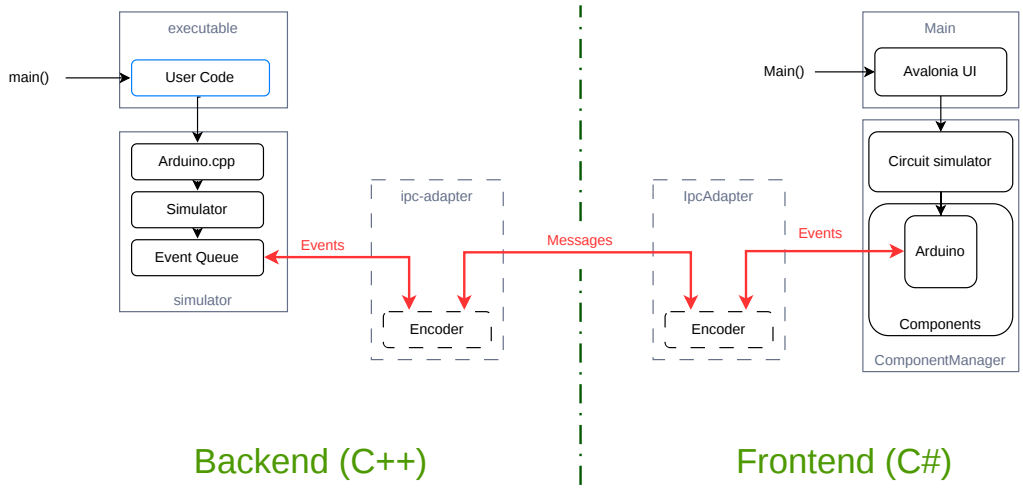

Figure 2.1 Top-level architecture overview

The Figure 2.1 shows the most important systems in each process. The user code is compiled together with the `executable` module of the backend. The declarations in the header `Arduino.h` prepended to the top of the `.ino` is implemented by a mock library `Arduino.cpp`. For each observable action an event is created and passed to the singleton `Simulator` class to compute and put into an event queue. The event is also communicated to the frontend process.

In the frontend the events are consumed by the `Arduino` component, one of several components predefined as part of the `ComponentManager` project. This project is responsible for loading user-defined components and scenes, which connect the components together in a circuit. The circuit is simulated in the same project. The user interface is implemented in the `Main` project using Avalonia UI as a framework. Interacting with the UI may invoke new events in the `Arduino` component. Those are sent back to the backend's event queue in the same manner as they were received.

The IPC Adapter is an interface implemented in each process responsible for encoding and decoding the events into a standardised format and relaying to the other process.

2.2 Inter-process communication

Implementing processes that depend on each other comes with certain challenges. Processes always run concurrently, so a variety of synchronization techniques must be applied to avoid race conditions. Before that we need to determine how will the processes exchange data. Process is a kernel level mechanism, thus approaches to handle the exchange can vary between operating systems. These mechanisms are what is called *Inter-process communication*, or IPC for short.

2.2.1 Considerations

There is a technical and an engineering reason for splitting the work into more than one process.

The user must be able to have full control over the frontend user interface at all times, even while the Arduino code is being debugged. By default, standard C++ debuggers operate in an "all-stop" mode where the target process must be completely paused to perform any meaningful interaction. Debuggers strictly refuse to display program states—such as local variables, memory contents, or call stacks—while any target thread is actively running, because the underlying memory and registers are constantly changing.

If the GUI has been part of the same process as the systems running the simulation, it would freeze and become unresponsive every time a breakpoint is hit. The only way to avoid this behavior is to separate the GUI to a different process. We want to support breakpoints in our project, as they are the most useful tool in a debugger's toolkit.

Splitting the application into multiple processes is actually a common practice among larger pieces of software. The program of each process can be written in a different programming language and thus delegated into teams with different competencies. They mark clear separation of concerns and force a solid interface design. Each process is easily swappable for another implementation with the same interface while the other processes are still running. In our case, the GUI process could be replaced by a web client, a CLI client, or an IDE add-on.

There are more than a few ways to handle IPC on every OS. The ones that we implemented are *TCP stream* and *shared memory*. These are among the most common approaches available both on Windows and Linux. In either case, we must clearly define what the shared data represent. Although shared memory has more flexibility than TCP, we stuck to sending discrete text or binary messages of predetermined format over time, to keep the implementation simpler and the two approaches interchangeable.

Another question is what will we encode in our message. At first, we encoded the full state of the simulated Arduino in *snapshots* but due to its various shortcomings we later switched to sending discrete *events*. So the simulator in the backend and its interpretation in the frontend both operate in terms of events, but TCP and shared memory modules only understand the notion of sending and receiving messages with very distinct interfaces. That's why we need to design an adapter in both processes and for both IPC methods that sets up the mechanism and encodes or decodes the passing messages into structured event data the simulation logic understands. The IPC adapter then informs the other

systems in its process any time a new event is received.

2.2.2 Snapshotting

In the early iterations, a snapshot of the Arduino state was sent over one message. That included the pinout state, pin modes, timer, etc. The state was periodically requested by the frontend with a *Read* message without payload. When a new value is read on an input pin in the frontend, a *Write* message with the state as payload would be sent to the backend and the values of input pins would be overwritten atomically to the backend's internal state.

This approach was easy to implement but flawed. The main issue was the frequency of reads. At lower frequencies, the system is prone to temporal aliasing. [1]Some events, like very short impulses, can be missed completely. This is not much of an issue in purely combinational circuits. But we want our frontend to support stateful components. For example shift registers, which are controlled by digital writes less than a microsecond apart. [2]

Sending snapshots can still be helpful, but not asynchronously, and not in set intervals as was the case in our approach. They could be sent occasionally from within the larger event system, in order to fix some cases of desynchronization.

2.2.3 Events

Right now synchronization between processes is maintained by recording every significant event produced by either the backend or the frontend, and sending it to the other process. Each process is equipped with an event queue, that consumes the events in order of the least recent to the most recent. Consuming an event means invoking the appropriate function based on the type of the event with arguments stored in the event's signature.

The Arduino object in the frontend has no logic on its own, so the only consumed events are those produced by the backend. For that reason a simple queue is sufficient, and no further synchronization is necessary. In the backend however, both simulator's and frontend's events are consumed. Sending an event to the frontend and receiving direct consequences of the event back can involve a significant time overhead. In the meantime the simulator produces more events that are consumed almost instantaneously. We need to ensure that the events are consumed in a valid order. We also need a robust queue implementation that isn't prone to race conditions.

An event is labeled with two timestamps. Timestamps don't have to be based on a real clock, the only requirement is that it is a non-decreasing sequence of numbers. The first timestamp sent is the simulation time, the time as perceived by the Arduino being simulated. That is time given by a clock that is started

[1]Aliasing is an effect better known in signal analysis, that occurs when the sampling frequency of a measured pattern is less than twice the frequency of the pattern itself. It makes false patterns appear in the sampled data. Temporal aliasing simply means that samples are taken in time intervals. In our case that could be, e.g., seeing LEDs blink at a wrong frequency.

[2]Highering the frequency to such an extent is not viable. It would mean taking a snapshot between every instruction of the Arduino code, which obviously takes much longer than the executed code itself.

right before the `setup()` is invoked. It can be manipulated by the user by pausing or slowing down. This is to aid the user while analyzing the logs of the simulator.

The other timestamp included is *Local virtual time* (LVT)[3]. Each process holds its own integer value of LVT. This number is increased with newly produced events and events received from the other process. When a decision has to be made on which event to consume next (like in the case of the backend's queue), the event with lower LVT is chosen. The relation $LVT_1 > LVT_2$ means that the event with LVT_1 happens after the event with LVT_2. In other words, event 1 is the consequence of the event 2 regardless of the real time passed between the events.

The rest of the information stored in an event is its type and arguments. The type corresponds to the action taken after the event is consumed, like writing to a pin or changing pin mode. Each type can define up to 3 integer parameters, e.g. for a *write* event that is ID of the pin, and a digital value we want to write to it. By limiting the number of arguments, we intend to avoid heap allocation of these events, which could considerably slow down the simulation with programs that produce a lot of events in a short time.

There are many ways to encode such an event into a message. Currently, the only encoder implemented in this work is the text encoder, which parses values of events into human-readable strings separated by a colon. The messages themselves are separated by a semicolon. This particular encoding is quite space ineffective, but it helps a lot during debugging of the IPC systems.

2.2.4 TCP stream

Messages can be sent and received through a stream of data on a TCP connection. TCP is commonly used to send data over a network, meaning backend and frontend processes can run on different machines. But listening to a connection on the same machine on the same port works just as well. TCP messages are guaranteed to arrive in the same order as they were sent.

The backend implements the TCP server, while the frontend implements the client. The implementations and design considerations behind these are very different based on the language they were written. But in both cases TCP handlers need to run on their own threads, due to how heavyweight they are. Listening to a network stream blocks the executing thread until new data are available. The data are sent as arbitrarily long sequences of bytes that need to be assembled into meaningful messages, and split up according to a specified delimiter. When a message needs to be sent from another thread, it is passed to the message handler where it is posted to the end of a queue and is scheduled to be processed by the OS.

There is a minor limitation of IPC over TCP that makes the debugging experience of the user's code uncomfortable. As we established, hitting a breakpoint will block all running threads of the process, including the thread with the TCP

[3]The terminology and principles are inspired by the *Optimistic synchronization* paradigm in distributed systems. In this paradigm, the events are consumed as fast as possible, and then rolled back once an event with a lower LVT than the ones consumed appears (so-called *Time Warp* mechanism). But Time Warp is not viable for simulating code of a high level language like C++ without custom memory management, and in our implementation it isn't even necessary. [5]

handler. When stepping through the lines of the code where events are produced, the TCP thread has to schedule posting of the message with the event, and it takes time for it to be eventually sent. In many cases the TCP thread won't be fast enough to send the message in the time the main thread progresses to the next line where a breakpoint will be hit again. As a result, consequences of the event won't be reflected in the frontend process until another one or more lines have been stepped through. This is the main reason we decided to also implement IPC with memory mapping, even though TCP is fully functional.

2.2.5 Shared memory

Memory mapping is the functionality of an operating system to relate a region of main memory 1:1 to a region in the address space of a process. This is an important idea in establishing a virtual memory of a process, but OS objects can be mapped as well. It could be either a *memory-mapped file*, contents of a disk file moved to the RAM. Or it could be a special OS object called *shared memory*, living in the kernel heap, specifically designed for IPC. In both cases, multiple processes can map the same memory region to their own address space. Thus they have access to the same data.

The terminology and API differ significantly between Linux and Windows, but with the use of the right libraries, the differences can be abstracted away and used as a single concept. So from now on we refer to the idea of mapping the same memory region to multiple address spaces as *shared memory*.

This tool is very powerful, we can address the shared data by pointers as we would with any other data of the process. There is also no need for system calls once the memory has been mapped. The changes are immediately visible to all processes that map the same region. As long as we do the writes in the simulating thread, we avoid the problem with delayed observations during debugging we encountered with TCP.

In the shared memory region, we establish a data structure that stores messages in a sequential order. We opted for a circular buffer to fully utilize the limited space of the region. There are actually two non-overlapping buffers in the same region together with a few bytes of metadata about their current state. A process consumes messages from one buffer while producing new messages to another. This is to avoid numerous race conditions that would occur while writing and reading from the same buffer while keep the synchronization mechanism lock-free.

As for the metadata, for each buffer a position after the last consumed and produced message is stored. It is important to never store references as pointers in the shared memory itself. The memory region will be most likely mapped to different addresses in each process. Because of that the *producer offset* and *consumer offset* are regular integers denoting the distance from the first address of the buffer in bytes. When the offsets are equal, it means the buffer is empty. We don't care about concurrent updates of these offsets, one process changes only one of the offsets for the duration of the simulation.

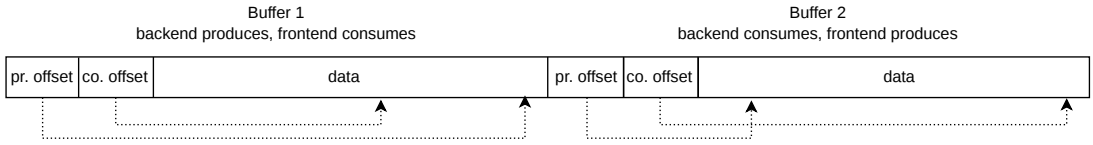

Figure 2.2 Memory layout of the shared memory region

2.3 Backend

2.3.1 The simulation

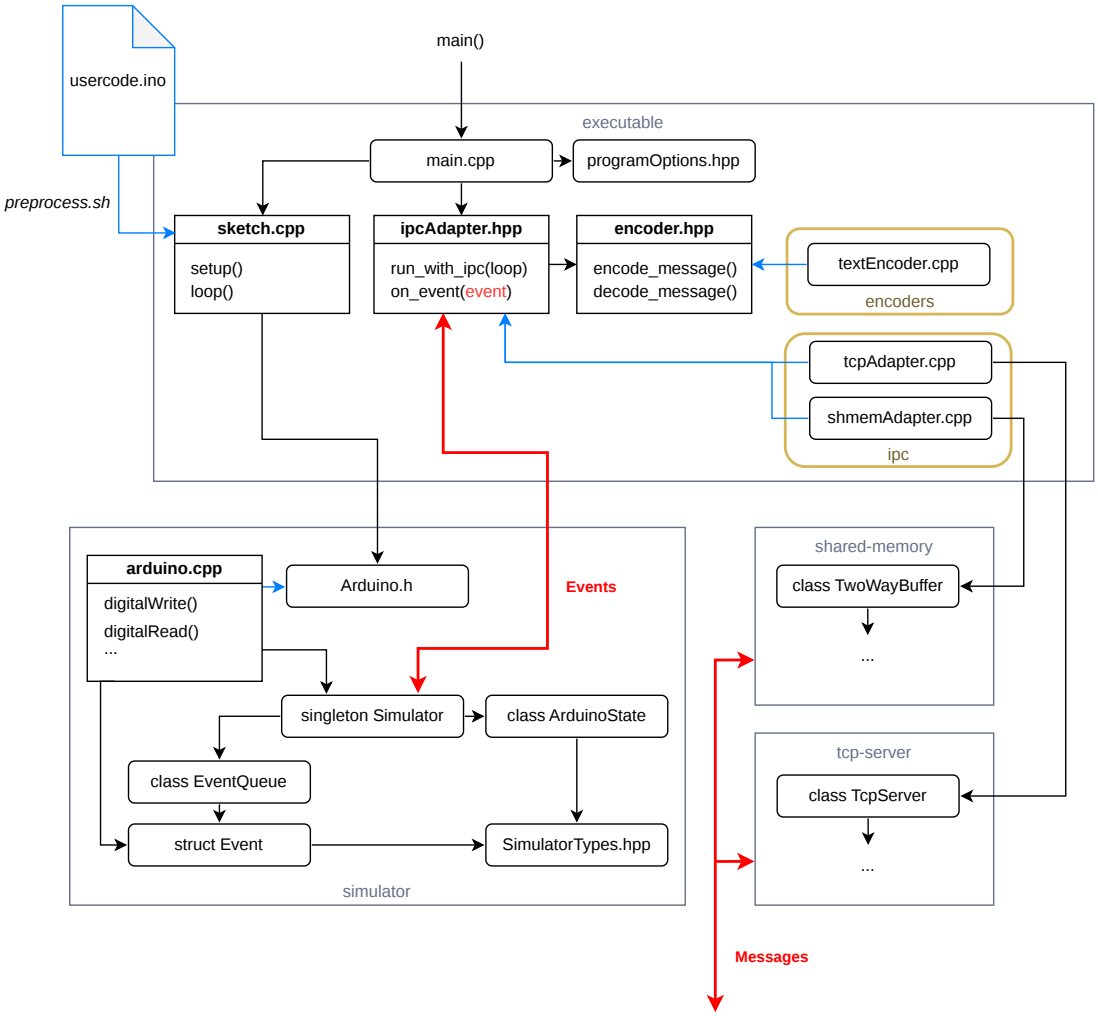

Figure 2.3 Backend architecture

The language choice for backend was obvious, the `.ino` file is a syntactically valid C++ code. We include the sketch source code in the project by passing it as an argument to the `preprocess.sh` script (`preprocess.bat` on Windows). Inside we use the `arduino-cli` tool with the right arguments to apply very light preprocessing to the file, like adding function declarations to the top or including the `Arduino.h` header. Important additions are the `#line` directives, which link to the definitions in the original `.ino` file. Thanks to these, breakpoints can be added to the `.ino` file and debugged as if the file itself was part of the project,

completely with stepping through lines and inspection of variables.

The file is saved in a new `sketch` directory in the `executable` module. Any source codes, like `.h` or `.c` files from the sketch are copied over to this directory. The preprocessed file, called `sketch.cpp` is inserted into this directory and included from `main.cpp`. CMake tracks the original sketch files as dependencies, and is instructed to use the preprocess script anytime their content changes.

The functions declared by the `Arduino.h` header file, like `digitalWrite` or `millis` are implemented in a custom mock library `Arduino.cpp` in the `simulator` module. Instead of instructing the AVR chip, as the real implementation would, the mock definitions create events and pass them to the `Simulator` for enqueueing and consumption.

The `Simulator` singleton class is the beating heart of the backend. It is responsible for handling the events and the event queue, managing the Arduino state, logging changes and keeping track of time. It is initialized in `main.cpp` and used frequently throughout the `executable` and `simulator` modules. The `Simulator` holds the instance of the `ArduinoState` class, representing the digital states on its pins, current pin modes and the inner timer of the Arduino. It also holds the instance of the `EventQueue`, which stores both local and remote events waiting for consumption. The `SimulatorTypes.hpp` is a header file with very few type definitions commonly used in the `simulator` module.

2.3.2 Events and IPC

The `Event` struct is just a "recipe" for what changes should the `Simulator` do to the `ArduinoState`. It holds a lambda function, corresponding to an event's type that is executed only after the event is put in the event queue and consumed in the order described in the section 2.2.3. The struct defines static functions that create new event instances with these predefined lambdas with the event type's parameters. When a local event is consumed, i.e. the lambda is executed, the event is signalled back to the `executable` module through a callback, that has been passed to the `Simulator` during initialization. This way the event can be sent to an IPC adapter, regardless of the used implementation, without a circular dependency on the `executable` module.

The IPC adapter itself is an interface of two functions declared in the `ipcAdapter.hpp`. `run_simulator_with_ipc` initializes the IPC mechanism. It takes the simulator loop function as an argument and executes it after IPC has been set up. That is because the IPC usually needs to create new threads that cannot be joined before the loop stops running. The loop function itself is defined in `main.cpp`. It contains both the `setup()` and `loop()` calls from the user's code.

The `on_event` function of the adapter is the callback passed to the `Simulator`. It encodes the event into a message and sends over IPC to the remote process. Listening to events of the remote process occurs on a different thread than the one with the simulator loop. Incoming messages are decoded into `Event` structs and sent directly to the event queue in the `Simulator`. `encode_message` and `decode_message` are functions declared in yet another interface in the `encoder.hpp` header. Its implementations are in the `encoders` directory, i.e. only `textEncoder.cpp` at the moment.

Implementations of the IPC adapter are located in the `ipc` directory.

The `tcpAdapter.cpp` depends on the `tcp-server` module, while the `shmemAdatper.cpp` depends on the `shared-memory` module. Which one is used depends on the chosen compilation target. The `tcp-server` module contains definition of the TCP server and mechanisms for accepting new connections and handling the stream of data using the `boost::asio` library. The `shared-memory` module contains memory mapping mechanisms using the `boost::interprocess`, implementation of the circular buffer and the producer-consumer pattern using two buffers.

2.3.3 Utilities

Logger

There are additional utility modules not shown in the figure 2.3. One of them is the humble `logger` used extensively throughout the project. The `ILogger` abstract class exposes the `log` function accepting either a raw string or an `IMessage` abstract struct. Structs inheriting form `IMessage` represent unified formatting of a message with a timestamp and a level of verbosity. The verbosity levels from the highest to lowest are *Debug*, *Info*, *Warning* and *Error*. The messages of higher verbosity levels can be omitted from the logs by setting the appropriate command line option. `ILogger` offers shorthand functions for logging these general messages, but more specific ones can be defined as well.

The module comes with `FileLogger` implementation of the `ILogger`. It creates or opens a file and dumps the logs inside. To avoid race conditions while writing to the files, two of these loggers are used in the project. One is owned and used by the `Simulator` singleton for logging inner workings of the Arduino. The other is owned by the IPC adapter and used for logging the exchanged messages or potential failures of the IPC mechanisms.

Timer

The other utility model is `Timer`. It is used to track time from the start of the simulation. The simulator holds two of these timers, one for *real time* and one for *simulation time*. Although there is no difference between them right now, the intention is to apply effects like stopping or slowing down to the simulation time timer from the frontend, so that functions like `millis` work consistently with disturbances caused by the frontend. Functions returning real time and simulation time are passed to the file logger as a source of its timestamps.

2.4 Frontend

2.4.1 The presentation

Our humble GUI is written using the Avalonia UI framework. It consists mostly of the circuit canvas, a zoomable and pannable space where the *components* of a circuit can be placed to specific coordinates, unconstrained by the usual layout structures of Avalonia. Arrangement of these components is called a *scene*. Each of the component types have its own set of SVG images that are drawn to the canvas. We refer to them as *sprites*. Each individual component always references

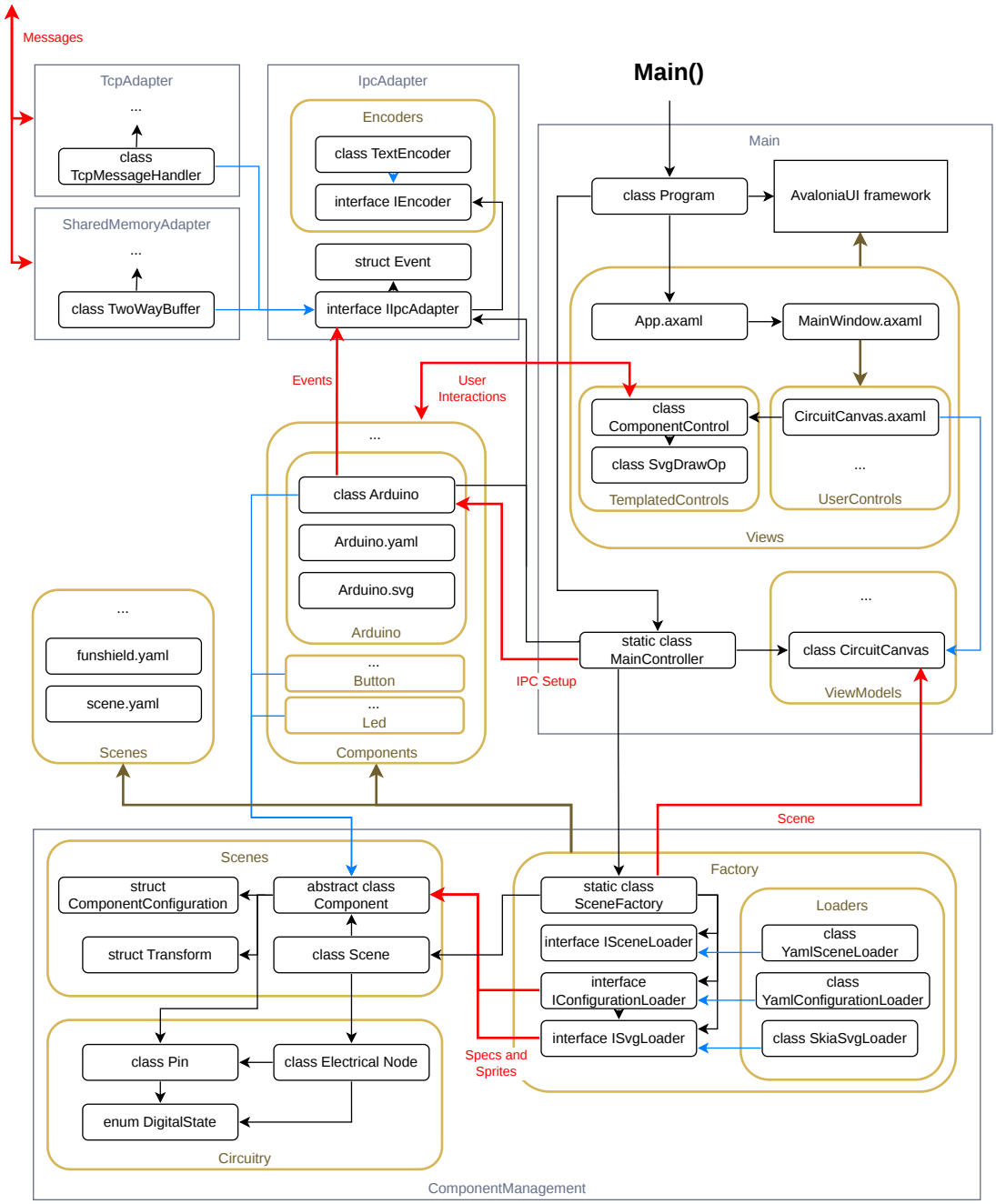

Figure 2.4 Frontend architecture

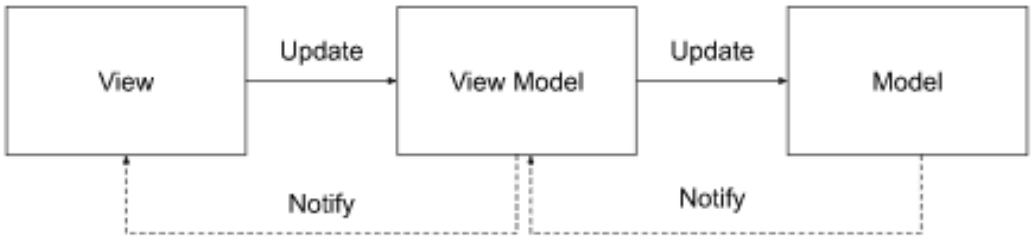

Figure 2.5 MVVM architecture [7]

one of its sprites, conveying the current state to the user. A sprite on the canvas can be clicked on (e.g. a button) to generate some kind of response. Of course, these features are not built into Avalonia, so we need to create custom Avalonia *controls* to achieve them. We only include Avalonia in the Main project of the frontend, as this is the only project responsible for rendering.

Model-View-ViewModel

The preferred architecture for Avalonia applications is *MVVM*, or *Model-View-ViewModel*. *View* is the visual layout of the window and its parts. It is written in *XAML* (Extensible Application Markup Language) in files with the `.axaml` extension. In Avalonia's case, elements of a view are called *controls*. Those can be buttons, labels, containers, etc. 3 types of custom controls can be created in total, our frontend utilizes *user controls* and *custom-drawn controls*. User controls are themselves views built up in XAML from other controls. This is perfect for our `CircuitCanvas` view, as it is just a list of component controls enveloped in a built-in `Canvas` panel. The custom-drawn controls are defined as C# classes, inheriting from the `Control` class, and defining its own rendering logic by overriding the `Render` method [6].

We need to implement the visuals of our components using custom-drawn controls, as there are no built-in or library controls that can draw a precompiled SVG image and swap it during runtime. Thus, we define a `ComponentControl`, that calls a custom drawing operation in its `Render` method. That is defined by the `SvgDrawOp` class, which takes a sprite and a transformation matrix as arguments, applies visual transformations on the image and instructs the Avalonia's rendering pipeline to show it in the parent view.

XAML views bind to data and events in a *viewmodel*, which manipulates with data shown in the view. A model then is the raw data passed to the viewmodel, accessed by some database or a service, or produced by the business logic. View is aware of the viewmodel and viewmodel is aware of the model, but not vice versa. In ideal case, the model should be agnostic about the UI technology used. [7]

Our model is the abstract `Component` class implemented by various types of components, like LEDs, buttons or our Arduino. A list of the `Component` instances is stored in `CircuitCanvas` viewmodel, and each instance is passed to a `ComponentControl` in the `CircuitCanvas` view. This and all other initialization steps of the application are orchestrated by the static `MainController` class.

2.4.2 The logic

Only a minor part of the frontend is actually written with Avalonia. The business logic of the aforementioned `Component` model is all encompassed in the `ComponentManagement` project. That means parsing of the components' definition, creating instances of a scene, and the simulation of the circuit. This project is completely unaware, that it will be rendered with Avalonia specifically.

The circuit can be thought of as an unoriented bipartite multigraph, where nodes of one partition are *electrical nodes* and nodes of the other partition are *components*. An edge of the graph represents a conductive connection between an electrical node and a *pin* of a component. Components usually have multiple pins.

In our digital circuit, an electrical node always obtains one of two digital states, *high* or *low*, which is given by states of the pins it is connected to. When a high state is written into a pin, we say it is *driving*. When a low state is written into a pin, it becomes not driving again. The digital state of an electrical node is high precisely when at least one of the pins it is connected to is driving. [4]A change in the state of an electrical node is propagated through all the connected pins. Components can react to this change by writing new states to their other pins creating a chain reaction of state changes in the circuit.

Components

The frontend's user is encouraged to extend the simulation with their own implementations of the abstract `Component` class. That is done by creating a new *component directory* with the name of the component. The shared path to all component directories is specified as one of the command line options of the frontend. The static information about the component type is stored as a record `ComponentConfiguration`, created from the files in the directory. The required files are a YAML file with metadata, a file with C# class, and at least one SVG file for the sprite. The YAML file contains in the very least a name to identify the component type, and a list of the component's pins. The pins can be defined either as an enumeration of their names, or as a single integer, that represents the total count of unnamed pins identified by their index. For full specification and an example of the configuration's YAML file, see subsection 5.2.2

The `Component` base class provides multiple virtual methods with the `On*` prefix. These are called upon changes in the circuit or as a result of a user interaction. The most important one is the `OnPinStateChanged` with the `Pin` object passed as an argument. This is the primary way of defining the component's logic. `OnControlPress` and `OnControlRelease` are other useful methods for interactive components, such as buttons. The base method `UpdateSprite` changes the rendered sprite to a different one, identified by the name of its SVG file. The base `GetPin` method returns a `Pin` object of the component's instance identified either by its name or index. Digital states are read and written through this object.

Usually, specific pins have inherent semantics of being used only for reading

[4]This is a simplification of how the term *driving* is usually used. *Drivers* are generally devices that force a state or signal on the electrical node. That could be both high or low digital states, or it could be e.g. a battery driving an LED. Driving both low and high state to a node may lead to short-circuiting, which is a situation impossible in our simulator. For more specific example, see how VHDL standard defines driving outputs. (Annex A.1, A.9 [8])

or only for writing. This property can be set explicitly on the `Pin` object with `MakeReadOnly` and `MakeWriteOnly` methods. Read only pins refuse any write attempts from the component's logic and write only pins refuse notifying a component with a new state from node's propagation. General pins, defined by method `MakeGeneral` don't have these constraints. Incorrect use of pins with these semantics is followed by a warning in the simulation logs.

Scenes

The final form of the graph is achieved with creation of the scene. The YAML file of the scene consists of two sections. In the `Components` section, individual instances of `Component` subtypes are defined. They must have a unique name, and they should specify a graphic transformation, i.e. position, rotation and scale. These values are composed into a `Transform` struct and passed into the `SvgDrawOp` as a transformation matrix. The `Nodes` section defines individual electrical nodes. A single node in this file is represented as a list of component-pin pairs, where the components are identified by their instance name in the `Components` section, and the pin is identified by its definition in the `ComponentConfiguration` of the given `Component` subtype. For further details see the subsection 5.2.4.

The creation of the scene is composed of a series of parsers in the `Factory` directory. There is an interface for each of the file types, i.e. the scene, component configurations and SVG files. Each has exactly one implementation in the `Loaders` directory, using `YamlDotNet` and `SvgSkia` libraries. They are put together in the static `SceneFactory` class, where component subtypes and instances are finalized. The results is a complete `Scene` object with initialized `Component` instances and `ElectricalNode` lists. The `SceneFactory` is called by the `MainController` in the `Main` project, which passes the scene to the `CircuitCanvas` viewmodel.

2.4.3 The Arduino component and IPC

Arduino is one of the component subtypes of the `ComponentManagement` project. It is a partial class divided into 2 files. In `Arduino.cs` it overrides virtual methods and deals with interaction with the circuit. In `Arduino.Events.cs` it creates events and sends to IPC. It also calls its own method from `Arduino.cs` when a new event arrives, based on its type and arguments. The adapter is now defined as an `IIpcAdapter` interface, which is owned by the Arduino component. Events are exchanged through this interface.

The interface is defined in the `IpcAdapter`, and is implemented by the `TcpAdapter` and `SharedMemoryAdapter` projects. It declares methods for connecting to IPC and sending messages. It also declares a C# event called `ReceivedEventEvent`, which is invoked after decoding a message incoming from the backend, with the decoded `Event` struct as an argument. The Arduino component is subscribed to `ReceivedEventEvent`.

The `IpcAdapter` project also contains message encoders and a definition of the `Event` struct. The C++ and the C# version of the `Event` struct have the same signature, with the exception of owning a lambda function (described in 2.3.2). The lambda is not necessary for the frontend, as there will be only one source of events executed in the frontend, the ones received by the backend. They don't have to be put in a queue, they can be consumed immediately as they come.

Frontend also produces events, e.g. when value on an Arduino's input pin is changed from the circuit, but those are never consumed by the frontend. It is the responsibility of the backend to handle them.

`IEncoder` and its `TextEncoder` implementation are analogous to `encoder.hpp` and `textEncoder.cpp` in the backend. The interface declares `EncodeEvent` and `TryDecodeEvent`, which transcribe events structs into human-readable strings and vice versa.

2.4.4 Utilities

Logger

The `Logger` project is analogous to its backend counterpart (2.3.3). It provides logging to a file with 4 levels of verbosity, that can be set up with a command line argument. Two loggers are used in the frontend as well, one for IPC and one for circuitry logs. There is one additional implementation of the `ILogger` called `CompositeLogger`, which holds list of loggers, and logs into all of them simultaneously. This mechanism anticipates a need for another logger implementation, which will display the log messages in the GUI using Avalonia.

3 Backend implementation

3.1 Simulator module

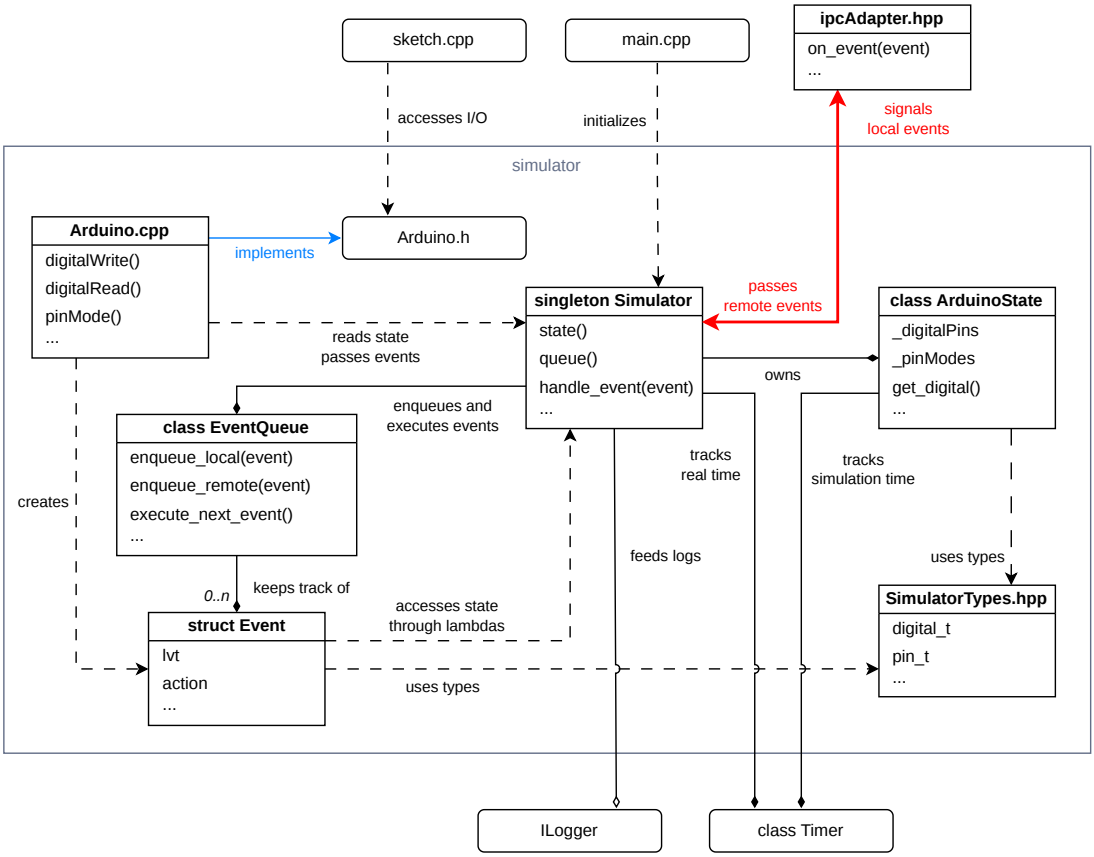

Figure 3.1 `simulator` module architecture

The `Simulator` class is initialized by the `main.cpp`, and its pointer is stored as a static field of the class itself, making it a singleton. It owns both the `ArduinoState` and `EventQueue` objects and provides access to them through its static classes. Namely, `handle_event` is called by the `Arduino.cpp`, passing events to the queue and consuming all events that are already waiting.

The events are created with static factory classes of the `Event` struct. In these, lambda functions are being created, utilizing the `Simulator`'s `state()` method, which provides direct access to the `ArduinoState`.

3.1.1 `ArduinoState` class

The class holds the digital states and modes of pins in `std::arrays`. Sizes of these arrays match the specification of the `Arduino UNO R3 SMD` board. The analog pins also stored in this class are not utilized at the point of writing this thesis. The pins can be read and modified with setters and getters analogous to those in `Arduino.h`, but instead of integers, the parameters are of types defined in `SimulatorTypes.hpp`. Namely `digital_t` is `bool`, `analog_t` is `double`, `pin_t` is `uint8_t` and `PinMode` is an enum attaining values `In` and `Out`.

The parameterless overloads return reference to an entire array. This was particularly useful when processing snapshots, before events were implemented. The setter methods return a bool indicating success of the operation. Currently, all methods always indicate success, but to achieve higher quality of life for users, warnings and errors should be logged instead of crashing the simulator immediately, e.g. when accessing pins outside of array's range.

The `_timer` owned by this class represents the simulation time. The `Timer` class is not described in detail in this thesis as it was originally designed for a different project, and a lot of its features are not interesting for this particular work. Under the hood it uses `std::chrono` library. The `deltaTime()` method returns microseconds passed from the timer's last step. The timers of this thesis are stepped exclusively at its initialization.

3.1.2 `Simulator` singleton class

The singleton's instance is created on the heap within the static `init` method and its pointer stored in the `s_instance` member. The `eventCallback` parameter of this method must be set to the handler, that should be called upon event consumption. In our implementation that is the `on_event` function from `ipcAdapter.hpp`. This callback is stored into `f_eventCallback`.

`state` and `queue` methods are used to access the objects owned by the `Simulator`. But the queue is more typically accessed through the `handle_event` and `handle_events` calls. They both consume all remote events waiting in the queue. The `handle_event` variant also puts a local event in the queue and consumes after all remote events with lower LVT.

The `Simulator` also owns a `Timer`, this one representing the real time passed. At the moment, both timers show roughly the same time, but it is anticipated, that the simulation timer could be manipulated by the user in the future versions of the project. Lastly, the `Simulator` class holds a shared pointer to a logger. It also exposes shortcut methods for passing log messages of different verbosity to the logger. This particular logger is used for any messages unrelated to the IPC.

`handle_event` method

Listing 3.1 `Simulator::handle_event` method

```
1  void Simulator::handle_event(Event event)
2  {
3      auto& queue = s_instance->_eventQueue;
4      queue.enqueue_local(event);
5
6      do {
7          queue.execute_next_event();
8      } while (event.id() != queue.last_executed_event().id());
9
10     s_instance->f_eventCallback(event);
11 }
```

The event is enqueued to the local event queue, and all remote events with lower LVT are consumed. To check that the local event has been consumed, we had to add a unique identifier member to the `Event` struct distinguishing different

event instances. This `id` is compared to the `id` of the previously consumed event, and the while loop is stopped when these two match. We couldn't use the LVT value as, even though it is unique for local events, it doesn't have to be unique across processes. In some frontend implementations, remote events can actually have higher LVT than the local ones.

Due to the semantics of LVT, we ensure that every remote event that occurred before the local event has been consumed. The exception are events that already occurred remotely, but haven't been fast enough to reach the backend's event queue. This can occur with TCP connection, or with very frequent `Arduino.h` calls in the sketch. These events will most likely be still consumed in the next call of `handle_event`. But for full correctness more synchronization techniques should be implemented, depending on the use case.

When the local event has finally been consumed, it is signalled to the IPC through the `f_eventCallback`.

3.1.3 Event struct

The members of the `Event` struct mostly match the event specification in the subsection 2.2.3. The additions are the `id` and `action` members. We already justified the use `id` member in the previous section. It is assigned in the constructor from the static `s_nextEventId`, which is then incremented. The `action` is assigned as a lambda function from within the static factory methods. This lambda is executed during the `EventQueue`'s `execute_next_event` call.

Event factories

Listing 3.2 `Event::write` function

```
Event Event::write(pin_t pin, digital_t value)
{
    Event event(Type::Write, pin, value);
    event.action = [=]() {
        Simulator::state().set_digital(pin, value);
        Simulator::log_info(std::format(
            "Written value {} to pin {}", value, pin));
    };
    return event;
}
```

The above is an example on how the factory methods should look like. The constructor takes a type and variadic arguments corresponding to that type. the constructor is statically asserted to not accept more than 3 variadic arguments. The action is defined as a lambda, with call to the `Simulator`'s `state` getter, modifying its values. The outcome is then logged through the `Simulator`'s logger.

At this point, the LVT of the event is not known. That is the responsibility of the scheduling entity. For local events it is the `EventQueue`, for remote events it is IPC, which has to decode LVT from an incoming message.

3.1.4 EventQueue class

The event queue is in reality two queues. One storing local and one storing remote events. As we only have two threads running concurrently in the backend,

this simplifies the synchronization significantly. We also completely avoid locking the structure in order to prevent race conditions. These queues are populated with the public `enqueue_local` and `enqueue_remote` methods.

`execute_next_event` method

Listing 3.3 `EventQueue::execute_next_event` method

```
void EventQueue::execute_next_event()
{
    auto& queueAhead =
        (earliest_queue_lvt(_localEvents) <
         earliest_queue_lvt(_remoteEvents))
         ? _localEvents
         : _remoteEvents;

    Event processedEvent = queueAhead.front();
    queueAhead.pop();

    processedEvent.action();
    _lastExecuteEvent = processedEvent;
}
```

Whichever event that has the minimal LVT of both queues has to be executed next. That is to preserve the semantics of LVT, such that every event with higher LVT is the consequence of an event with lower LVT. We expect that the queues are sorted by the LVT of their events, from lowest to highest. For local events that is given by sequential execution of the sketch. For remote events this is given by the IPC technologies we've chosen. TCP guarantees that data are received in the same order as they were sent. In the shared memory approach, we deliberately write and store messages in a sequential order. So to determine the next event, all we have to do is peek at the first element of both of the queues, and select the one with lower LVT. That is ultimately how `queueAhead` is obtained.

The event is then popped from the queue. The `action` lambda stored within it is finally executed. The event can now be considered consumed. Then the `_nextLvt` is decided. If the consumed event has the highest encountered LVT yet, we simply increment this value. If it does not, we don't change the value, as it would mean the subsequent events would technically occur in the past.

3.1.5 `Arduino.cpp`

Listing 3.4 `Arduino.cpp`

```
using namespace arguino::simulator;

void digitalWrite(uint8_t pin, uint8_t val)
{
    if (Simulator::state().get_digital(pin) == val) return;

    Simulator::handle_event(
        Event::write(pin, val == HIGH)
    );
}

```

```
int digitalRead(uint8_t pin)
{
    Simulator::handle_events();
    return Simulator::state().get_digital(pin);
}

unsigned long millis()
{
    return std::floor(Simulator::state().get_time() / 1000);
}

void pinMode(uint8_t pin, uint8_t mode)
{
    PinMode pinMode = mode == OUTPUT ? PinMode::Out : PinMode::In;
    if (Simulator::state().get_pin_mode(pin) == pinMode) return;

    Simulator::handle_event(
        Event::set_pinmode(pin, pinMode)
    );
}
```

At last, we implement the four functions specified in the goals of the thesis. `millis` simply takes the simulation time and divides it by 1000 to get the elapsed milliseconds. The `digitalWrite` and `pinMode` functions are implemented using the `handle_event` call with the corresponding event factory passed as an argument. If the argument for these events doesn't differ from the current value stored in `ArduinoState`, we exit the function early. In sketches with these functions in the outermost `loop` scope, which is actually a common pattern, the condition can save millions of redundant events and log messages.

The `digitalRead` call consumes all waiting events, and then simply reads the pin of `ArduinoState`. Out of the four functions, this is the only one that can be problematic with regard to delayed event reception described in the `handle_event` method (3.1.2), as it is the only one dependent on previous `Arduino.h` calls. It is possible for `digitalRead` to return a state outdated by a few microseconds.

A proposed solution is to add an event serving as a kind of condition variable. This event would be sent to the frontend and the same event would be sent back after any necessary frontend changes would have been resolved. Until then, the backend's simulation thread would remain in a spin lock, pausing the simulation timer, handling remote events and checking for the expected synchronization event. Only then the function would return a value and the execution of the sketch would resume.

This proposal hasn't been implemented at the time of writing the thesis due to time concerns. This incorrect behavior hasn't been yet observed with the provided tests, and it probably won't be noticed on simpler sketches of a target user audience.

3.2 Executable module

The `executable` module puts all the systems together and compiles into an executable file. `sketch.cpp` doesn't come with the source code. It has to be preprocessed from the sketch user wishes to simulate. `main.cpp` defines how the

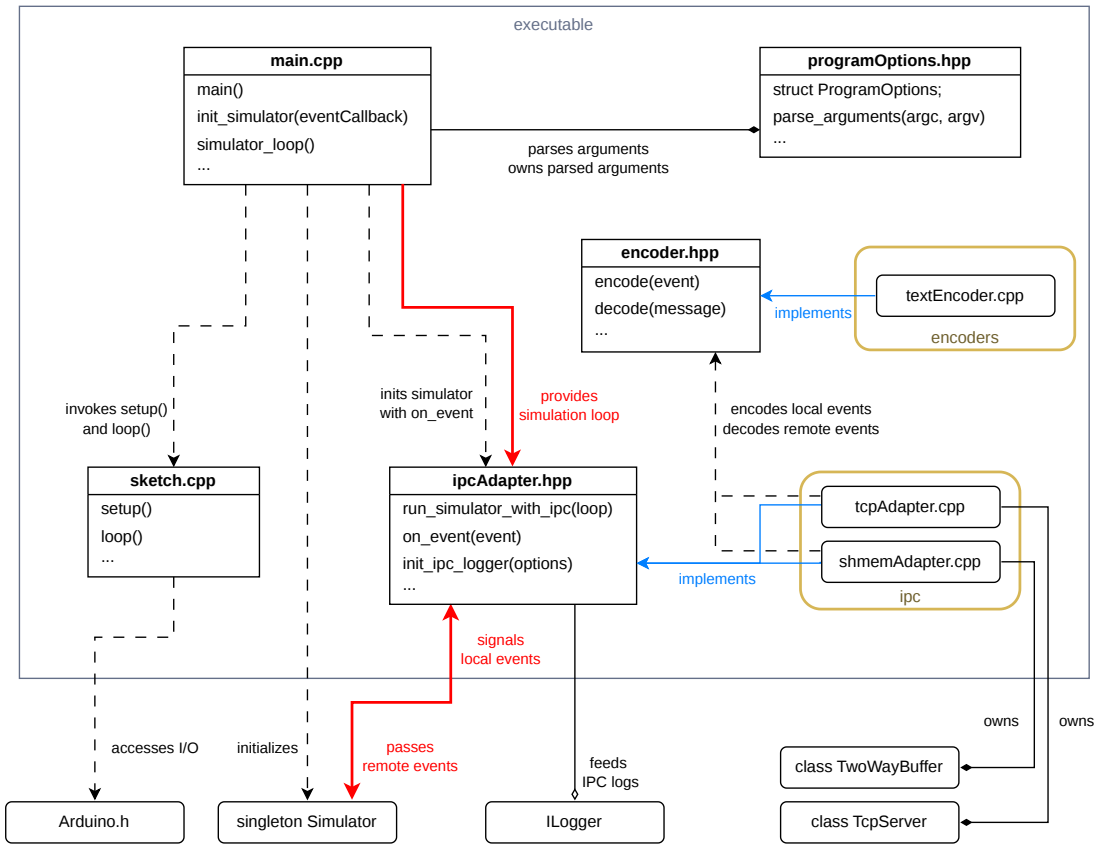

Figure 3.2 `executable` module architecture

sketch is used, and orders the IPC adapter to use it in context of the used IPC mechanism.

Two execution targets are provided in the CMake configuration, one for each IPC mechanism. They differ in which IPC adapter implementation is used, and which program options are supported with that implementation. They also differ in whether the `tcp-server` or the `shared-memory` module is linked with the executable.

3.2.1 `main.cpp`

Listing 3.5 `main` function

```
1 int main(int argc, char** argv)
2 {
3     options = parse_arguments(argc, argv);
4     init_simulator(on_event);
5     run_simulator_with_ipc(simulator_loop, options);
6 }
```

The `main` function of the C++ project parses the command line arguments, initializes the Simulator singleton, defines the simulation `loop` and lets the IPC adapter handle the rest. The `on_event` function passed to `init_simulator` is included from `ipcAdapter.hpp` and implemented by the adapter chosen via CMake execution target. Simulation logger and it's verbosity are created in the `init_simulator` as well.

The `simulator_loop` calls the the required functions included from `sketch.cpp`: the `setup` function once and the `loop` function in an infinite while loop. The *Reboot* event is sent as the very first event of each simulation. This is useful when the frontend keeps running, but backend is being restarted or recompiled with a different sketch. The lack of such an event would preserve the frontend state of the old simulation, and it would get desynchronized very quickly.

The command line arguments are parsed using the `boost::program_options` library. The values in their expected data types are stored in the `ProgramOptions` struct. An instance of this struct is stored statically in the `main.cpp` file, and passed down to the IPC adapter by reference.

3.2.2 `ipcAdapter.hpp`

The responsibility of the `run_simulator_with_ipc` method is to initialize the classes used for IPC and store them in the implementing `.cpp` file. That typically involves creating a new thread for listening to messages incoming from the frontend process. This thread has to be joined at some point after the simulation loop. That is why we pass the loop function as an argument, to be invoked at the right time.

The `on_event` function is passed to the `Simulator` and is called every time a local event is consumed. The event is encoded into a message using the function included from `encoder.hpp`. The message can be then transmitted over the IPC, which has already been initialized before the very first local event is created.

The `init_ipc_logger` function is already defined as it provides the IPC logging setup common for any adapter. We call this function at the beginning of each `run_simulator_with_ipc` definition.

Implementations are located in the `ipc` directory of the `executable` module. But we should first understand how were the IPC mechanisms implemented, so we moved the adapter documentations to their respective chapters (subsections 3.3.3 and 3.4.4)

3.2.3 `textEncoder.cpp`

The `encoder.hpp` declares a function that encodes local events into messages, and decodes messages into remote events `decode_event` outputs the result event as a reference parameter. Its return value indicates success of the operation. That is, whether was the received message in a valid a format. The `UNKNOWN_EVENT` constant is used to identify undefined event types. [1]

At the time of writing this thesis, `textEncoder.cpp` is the only encoder implementation. It handles human-readable messages, the format of which is described in more detail in the section 5.3.1. The message is divided into 3 to 6 parts delimited by a colon, the first 2 of which are common for every event type. Hence, the constant string formats for each of the types. `encode_message` identifies the type, fills the event values into the respective format, and its done.

[1] This can occur when implementing new events, but forgetting to add new cases in encoding. This should never happen with the source code attached with this thesis.

Decoding a message

Listing 3.6 `decode_event` implementation

```
bool decode_event(const std::string& message, Event& event)
{
    parts_array_t parts = split(message);
    std::array<int, MAX_ARGS> args;

    for (int i = 0; i < MAX_ARGS; ++i) {
        if (!decode_int(parts[COMMON_PARTS + i], args[i])) break;
    }

    if (parts[2] == "W") {
        event = Event::write(args[0], args[1]);
    }
    else if (parts[2] == "P") {
        event = Event::set_pinmode(args[0], (PinMode)args[1]);
    }
    else {
        return false;
    }
    if (!decode_int(parts[0], event.timestampMicros) ||
        !decode_int(parts[1], event.localVirtualTime)) {
        return false;
    }
    return true;
}
```

`decode_message` has to manually split the message into substrings and parse each substring individually. First we decode arguments in a loop. Then we decode the type encoding character, and based on that we call the appropriate factory method with the right arguments. In the end we decode the timestamps from the common parts.

At each of these parts we can encounter a value that doesn't conform to its expected data type or a known event type, and the decoding fails. Then we have to return `false`. At this point, the event can be already partially initialized, but it is the responsibility of the caller to discard this instance, and fall into some fail state.

3.3 TCP Server module

IPC over TCP is implemented using the `boost::asio` library. It serves as an abstraction over asynchronous I/O and networking resources of the OS. The core class of the library is the `io_context` type, in which the network operations are scheduled and executed. The `tcp::acceptor` type attaches to the context and listens for any TCP connection on the given endpoint (ip and port). At that time `ConnectionHandler` instance is created. This happens within the `TcpServer` class.

Once a connection has been established, it connects to a socket owned by the `ConnectionHandler`. Inside its `handle` method, the connection handler waits asynchronously for any incoming data. Once data are available, and the message delimiter is found within them, the data is passed to the `handle_message` where

it is preprocessed and the passed to a handler callback passed to the class from the TCP adapter.

If a message needs to be sent over TCP, it is first passed to `TcpServer` and that passes it to the `ConnectionHandler`. It is put to an outbox queue and sent asynchronously over the same socket, within the same `io_context`. [2]

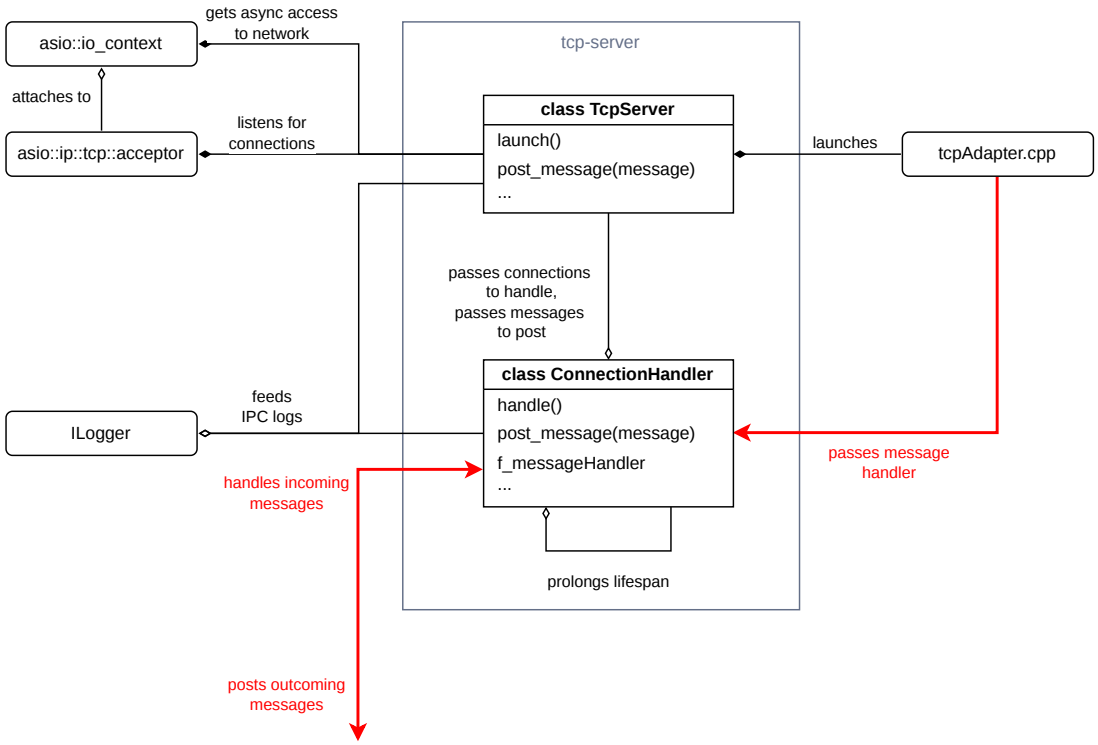

Figure 3.3 `tcp-server` module architecture

3.3.1 `TcpServer` class

The class initializes the `io_context`, the `acceptor` and the `endpoint` within its constructor. One of the contructor's arguments is callback, that has to be called once a message is received by the `ConnectionHandler`. We expect only one client to connect at a time. [3]We store a shared pointer to this handler and pass it the `_messageHandler` callback once it is ready. Local messages that need to be sent to the frontend are also passed to the same `ConnectionHandler`. If there is no connection, sending a message blocks the calling thread, hence the `_connectionCv` condition variable.

Accepting connections

[2]The general outline of the server and connection handler is taken from `boost::asio` tutorials [9]. A CppCon presentation by Michael Caisse [10] was incredibly helpful for practical understanding of the library and common patterns.

[3]A more robust architecture would of course allow multiple clients to be connected at once, which would require to decouple handler from the server. But only one connection also means we don't need any thread pool to handle the traffic.

Listing 3.7 `launch` method

```
void TcpServer::launch()
{
    _acceptor.listen();
    start_accepting();
    _ioContext.run();
}
```

The server starts to listen for connections by calling the `launch` method. Within it the acceptor starts to listen for connection with the `listen` method. What must happen after a connection is established is defined by the `start_accepting` method. `_ioContext.run()` blocks the calling thread and starts executing any asynchronous operations that have been scheduled for execution. The method return once there is no work to be done, so we have to make sure there is at least one scheduled operation.

Listing 3.8 `start_accepting` method

```
void TcpServer::start_accepting()
{
    auto newConnection = handler_t::create(
        _ioContext, _messageHandler, _logger);

    _acceptor.async_accept(
        newConnection->socket(), [=, this](auto error) {
            if (!error) {
                _logger->log_info("Connection accepted!");
                _connectionHandler = newConnection;
                _connectionCv.notify_one();
                _connectionHandler->handle();
            }
            else {
                _logger->log_error("...");
            }
            start_accepting();
    });
}
```

Async TCP operations generally require a reference to a `tcp::socket` object and a callback to call once the operation has been executed by the `io_context`. A socket is always associated with a single connection, so we define it in our `ConnectionHandler` class and create its instance beforehand.

In the callback we notify the `_connectionCv` that a connection has been found, store the pointer of the handler and call its `handle` method. Then we must call the `start_accepting` again. This is another common pattern among async operations. It makes sure that another async operation is scheduled immediately after the first one has been completed, listening for new connections, messages, etc. Without them `io_context` simply stops. This looks like a recursive pattern, but remember that the outer function has finished long before the callback is invoked by the executing context.

3.3.2 `ConnectionHandler` class

The `ConnectionHandler` owns the socket, used for asynchronous TCP operations. It provides the public `handle` and `post_message` methods for scheduling

reading and writing operations respectively. The private `handle_message` and `write_from_queue` are used within the callbacks of these operations.

The `ConnectionHandler` inherits the `enable_shared_from_this` with itself as the template argument. This allows us to call the inherited `shared_from_this`, that creates a new shared pointer, pointing to itself. Those are used to prolong the lifetime of the object with every subsequent scheduled operation, as each one of the creates a new pointer. This pattern is very common in ASIO servers, especially when the handlers are not owned by any higher entity.

Receiving messages

Listing 3.9 `handle` method

```
void ConnectionHandler::handle()
{
    _isConnected = true;

    boost::asio::async_read_until(
        _socket,
        boost::asio::dynamic_buffer(_incomingBuffer),
        MESSAGE_DELIMITER,
        [me = this->shared_from_this()]
        (auto error, size_t messageSize) {
            if (error == boost::asio::error::eof) {
                me->disconnect();
            }
            else {
                me->handle_message(error, messageSize);
                me->handle();
            }
        }
    );
}
```

Upon new connection, the handler starts immediately listening for new messages with the `async_read_until` function. Aside from usual arguements it requires a delimiting character (semicolon in our case) and an `asio::buffer`. Buffers in the context of ASIO are views of contiguous regions of memory. We use a dynamic buffer (meaning it can change size) and back it with the `_incomingBuffer` string.

The method reads data from the network until the delimiting character has been encountered. Then the portion of the data until the delimiter is copied into the buffer. And after that the callback is called with the number of copied character passed as an argument. We process the data in the buffer with `handle_message` method.

Listing 3.10 `handle_message` method

```
void ConnectionHandler::handle_message(
    boost::system::error_code error, size_t messageSize)
{
    if (error) {
        _logger->log_error("...");
        return;
    }
```

```
8
9       if (_incomingBuffer.empty()) return;
10
11      const std::string message =
12          _incomingBuffer.substr(0, messageSize - 1);
13      _logger->log_debug("...");
14      _messageHandler(message);
15
16      _incomingBuffer.erase(0, messageSize);
17  }
```

We check for errors and strip the message of its delimiter. Finally, we pass the message to the `_messageHandler` which was bestowed to us from the `tcpHandler.cpp` through the server.

Posting messages

Listing 3.11 `post_message` method

```
1  void ConnectionHandler::post_message(const std::string& message)
2  {
3      {
4          std::unique_lock lock(_queueMutex);
5          _queueCv.wait(lock, [this]() {
6              return _outboxQueue.size() <= MAX_QUEUE_SIZE;
7          });
8
9          _outboxQueue.push(message);
10     }
11
12     boost::asio::post(_executor,
13         [me = this->shared_from_this()] {
14             me->write_from_queue();
15         });
16 }
```

For scheduling messages to send we use a queue of strings called `_outboxQueue`. Since this method is most likely going to be called from a thread different that one of `io_context` we need to wrap the sending logic in `post` function. This schedules the `write_from_queue` method in the execution context of the `io_context` thread. The `_executor` argument represents this. It has been retrieved from `io_context` in the connection handler's constructor.

We set the maximum count of messages in the queue, so that the simulator doesn't do unnecessary work if it isn't listened to. Once this count has been reached, the calling thread is blocked with the `_queueCv` condition variable.

Listing 3.12 `write_from_queue` method

```
1  void ConnectionHandler::write_from_queue()
2  {
3      if (_isWriting) return;
4      if (_outboxQueue.empty()) return;
5
6      _isWriting = true;
7
8      boost::asio::async_write(_socket,
9          boost::asio::buffer(_outboxQueue.front() + ';'),
```

```
        [me = this->shared_from_this()]
        (auto error, size_t bytes_written) {
            if (error) {
                me->_logger->log_error("...");
            }
            else {
                me->_logger->log_debug("...");
                me->_outboxQueue.pop();
                me->_queueCv.notify_one();
            }

            me->_isWriting = false;
            me->write_from_queue();
        }
    );
}
```

The messages are sent with the `async_write` operation. It requires a buffer from which the data will be copied to the network. We back this up by one element of the `_outboxQueue` with appended delimiter. Once succeeded, the message is removed from the queue and the condition variable is notified with the change of the queue's size.

It is possible for the `write_from_queue` method to be invoked twice before `async_write` executes even once. This would end up by sending the same message multiple times. So we guard the method with `_isWriting` bool, that is set to true once the method is called, and only set to false after the `async_write` has been executed. This causes that the number of `post_message` calls doesn't match the number of `async_write` calls. So we call the method repeatedly from the callback, such that we send all the messages waiting in the queue.

3.3.3 tcpAdapter.cpp

Listing 3.13 `on_received_tcp_message` function

```
static void on_received_tcp_message(const std::string message)
{
    Event event;
    if (decode_event(message, event)) {
        Simulator::queue().enqueue_remote(event);
    }
    else {
        IpcLogger->log_error("...");
    }
}
```

The implementation of `tcpAdapter.cpp` is quite simple. The `TcpServer` is instantiated with `on_receive_tcp_message` as its message handler, and its `launch` method is invoked in a new thread. Then simulation loop starts.

`on_event` and `on_receive_tcp_message` are semantically opposite to each other. The former encodes an event into message and enqueues it to the server. The latter decodes a message into an event in enqueues it to the `Simulator`.

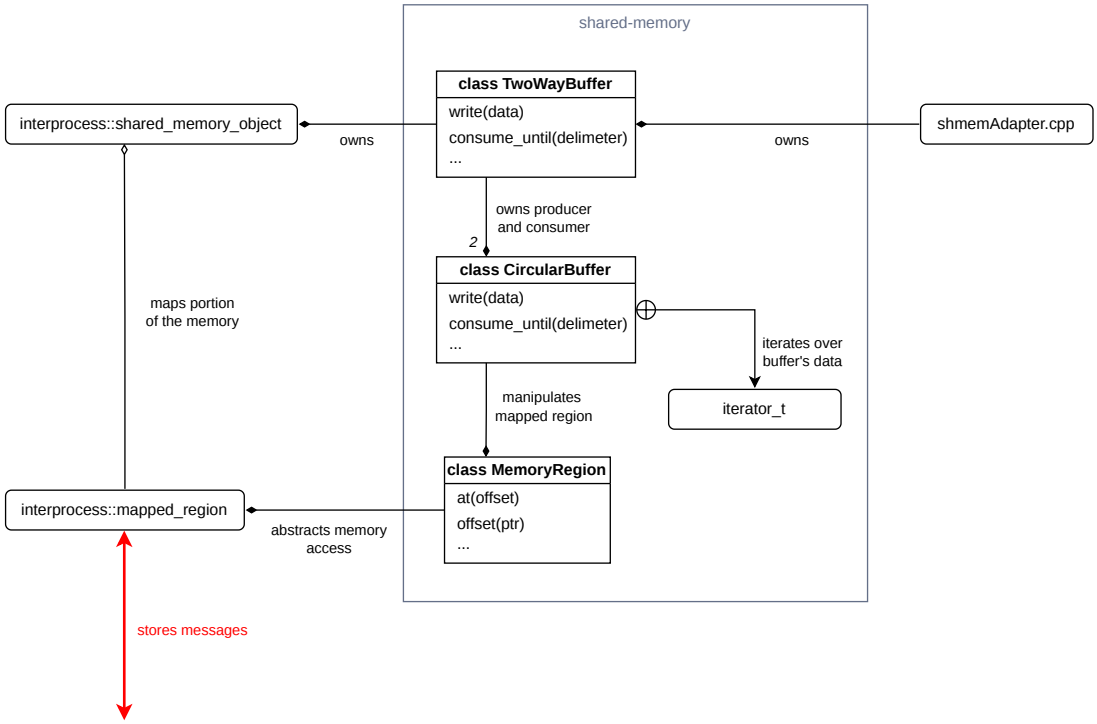

Figure 3.4 `shared-memory` module architecture

3.4 Shared Memory module

The `shmemAdapter.cpp` holds a `TwoWayBuffer` object. It represents the two circular buffers, one producing and one consuming. It creates the OS-level memory mapped object using the `boost::interprocess` library, and splits it into two regions of the same size, one for each buffer.

The `CircularBuffer` class implements the data structure of the buffer itself. It was designed deliberately for use with shared memory. It holds the `MemoryRegion` instance, which abstracts away the low level details of accessing the shared memory directly. We defined a forward iterator for the `CircularBuffer` to simplify the manipulation of its actual data.

3.4.1 `MemoryRegion` class

This class provides an abstraction over the `interprocess::mapped_region` type. The aforementioned type isolates a region of memory within the `interprocess::shared_memory_object` and provides direct access to it. We create the `mapped_region` object in the constructor, defined by its size and the offset from the starting address of the `shared_memory_object` in bytes. We set its permission to read and write.

The methods of `MemoryRegion` consist mostly of transformations from `void*` to `uint8_t*`, and conversions between pointers and offsets. We cannot perform pointer arithmetic on `void*`, so it is useful to have the pointers already in form that allows passing through memory byte by byte. We already mentioned the importance of offsets in the section 2.2.5.

The templated `at` method uses reinterpretation casting to automatically re-

trieve the underlying type from raw bytes, and returns a reference to it.

3.4.2 `CircularBuffer` class

The data structure occupies the entire space defined by the underlying `MappedRegion`. The producer and consumer offsets occupy the first 16 bytes of the region. The rest is filled with the raw data, simply referred to as `buffer` in the code. In the context of this structure, the offset must be interpreted as the distance from the address from the start of the buffer.

To simplify this notion we defined a forward iterator with this offset as the underlying iteration unit. It dereferences into `uint8_t`. Incrementing over the boundary of the memory region sets the offset back to 0. `producer_it` and `consumer_it` turn the offsets stored in the raw memory into these iterators. They can be written back with the `write_producer` and `write_consumer` methods.

For the purpose of writing and reading messages (or bigger structures in general), we define the `write` an `consume_until` methods. These implicitly access the data at `producer_it`, respectively `consumer_it`, onwards. `write` comes in two variants, one for contiguous ranges and one for any other templated type. The return value indicates whether or not does the buffer have enough available space to insert the type. `consume_until` takes a delimiter as the point at which to stop reading the data. It returns the read bytes in an `std::vector` and moves the producer offset.

Writing to buffer

Listing 3.14 `CircularBuffer::write` method

```
template <std::ranges::contiguous_range TRange>
inline bool CircularBuffer::write(const TRange& data)
{
    auto bytes = std::as_bytes(std::span(data));

    // We don't want the buffer to be fully filled
    if (bytes_available() <= bytes.size()) {
        return false;
    }

    size_t bytesUntilBufferEnd = end() - producer_it();

    if (bytes.size() < bytesUntilBufferEnd) {
        std::memcpy(
            &*producer_it(), bytes.data(), bytes.size());
    }
    else { // Message written on a boundary
        auto firstBytes = bytes.first(bytesUntilBufferEnd);
        auto lastBytes = bytes.last(
            bytes.size() - bytesUntilBufferEnd);

        std::memcpy(
            &*producer_it(), firstBytes.data(), firstBytes.size());
        std::memcpy(
            &*begin(), lastBytes.data(), lastBytes.size());
    }
```

```
27
28      write_producer(producer_it() + bytes.size());
29      return true;
30  }
```

The range variant of `write` is the one that does all the hard work. The other one just transforms the type `T` into an `std::span` with the constant reference to `T` as it's only element. This qualifies it as a range.

First we turn the range into a span of bytes, so we can access each one of them using pointer arithmetics. Then we check if there is enough available space to store all the bytes. We have to make sure that there is always at least one byte unused. Otherwise, would the consumer and producer offsets equal. We specifically interpret this state as the buffer being empty. Without this specification it would be ambiguous whether is the buffer empty or full.

Then we check if there is enough space left until the end of the buffer. If not, we will have to write the data on its boundary by splitting it in two. The `firstBytes` get copied to the producer offset, and the rest at the offset 0.

To update the producer offset, we need to use atomic instructions to write it back to the memory. This is because the compiler can make optimizations that would reorder the reorder or omit memory access instructions. It could possibly order writes to the buffer's main memory to be executed after write to the producer offset. It could also leave the offset in an invalid state, as normal store is not an atomic operation. We use the `std::memory_order_release` as an argument for the atomic store, which prevents any prior read and write from being reordered after this atomic store.

Listing 3.15 `write_producer` method

```
1  void CircularBuffer::write_producer(iterator_t producer_it)
2  {
3      std::atomic_ref<uint64_t> atomic_producer(
4          _memoryRegion->at<uint64_t>(PRODUCER_PTR_LOCATION));
5
6      atomic_producer.store(
7          producer_it.offset, std::memory_order_release);
8  }
```

Reading from buffer

First we need to copy the producer offset as it can change during the execution of the function. Reading the offset must also be an atomic operation, for the same reason as writing. The compiler could use optimizations that lead to undefined behavior. Since `iterator_t` is a fully qualified forward iterator, we can pass the offset iterators as arguments of `std::find` in order to find the iterator of the delimiter. If the delimiter hasn't been found, we return an empty vector, indicating that the operation doesn't make sense with the current state of the buffer.

Listing 3.16 Finding delimiter using the `std::find` function

```
1  std::vector<uint8_t> CircularBuffer::consume_until(
2      uint8_t delimiter)
3  {
4      iterator_t producerIt = producer_it();
```

```
    iterator_t consumerIt = consumer_it();

    auto delimiterIt = std::find(
        consumerIt, producerIt, delimiter);

    bool delimiterNotFound = delimiterIt == producerIt;
    if (delimiterNotFound) return {};
    ...
}
```

Otherwise, we proceed analogously to `write`. We determine the length of the message, and if it could fit until the end of the buffer. If not, it lies on the boundary, so we have to copy portion of the data from the end of the buffer, and portion from its beginning.

3.4.3 `TwoWayBuffer` class

`TwoWayBuffer` is a wrapper around the `CircularBuffer` class. It owns the instances of both buffers, and limits access to them. `write` passes its arguments to `write` method of the producer buffer and `consume_until` passes its argument to the method of the same name of the consumer buffer.

The class also creates and owns the `shared_memory_object`. This object either creates a new OS-level shared memory object, or opens one with the given name if it already exists. Opening the object means it doesn't need to be destroyed, and the frontend doesn't need to shift into a panic state. At the construction of the circular buffers, both of the offsets are reset 0, effectively clearing any previous data.

The memory object needs to be specified its size, which is always a multiple of the page size on the given OS (usually 4096B). `TwoWayBuffer` then dedicates half the size of the memory object to each of the circular buffers.

3.4.4 `shmemAdapter.cpp`

Listing 3.17 `run_shmem_consumer` function

```

static void run_shmem_consumer()
{
    while (true) {
        while (!_shmem->consumer().is_empty()) {
            std::vector<uint8_t> data = _shmem->consumer()
                .consume_until(
                    MESSAGE_DELIMITER);

            if (data.empty()) {
                IpcLogger->log_error("...");
                continue;
            }

            std::string message(data.begin(), data.end());
            IpcLogger->log_debug("...");

            Event event;
            if (decode_event(message, event)) {
```

```
                    Simulator::queue().enqueue_remote(event);
                }
                else {
                    IpcLogger->log_error("...");
                }
            }
        }
}
```

The adapter utilizes the shared memory through the high level `TwoWayBuffer` wrapper. It provides it its name and size in pages from options. Then it creates a single thread that infinitely checks for new messages in the consumer buffer. Instead of the spin lock (inner while loop in `run_shmem_consumer`) it would be more appropriate to use interprocess semaphore to check the contents of the buffer, saving some resources. Inside the loop a message delimited by a semicolon is retrieved, decoded and enqueued. `on_event` method is as simple as encoding the event and writing it to the buffer, given there is enough space.

4 Frontend implementation

Note on the `event` keyword

Until now we've been talking about events exclusively in the context of IPC. But since C# defines it's own `event` keyword, used multiple times in the frontend's code, using the two different meanings could become confusing. So for the duration of this chapter, we will write *event* (in italics) every time we talk about the C#'s keyword and the associated patterns. The notation of the IPC event and `Event` structs remains unchanged.

4.1 Component Management project - Circuitry and Scenes

Classes of the `Circuitry` and `Scenes` directories are a tightly coupled system, encapsulating the core logic of the frontend. The `Scene` classes consists of nothing more than a list of components and a list of electrical nodes. Components of the same electrical node interact with each other through the interface of pins. These can either listen to changes of digital state on the node, or they could assert a high state to it (driving).

`Component` is the class overridden by user in order to create a custom component. It holds `ComponentConfiguration` structs statically in a dictionary. This struct contains the metadata of a `Component` subtype, available sprites and pin identifiers. Each `Component` instance owns a set of `Pin` instances orchestrating the circuit's logic.

One of the `Component` subtypes included in this project is the `Arduino` class. It is the component that connects from IPC to the backend simulator. Since the component is so important, and it depends so heavily on the `IIpcAdapter` (it is basically the frontend for the adapter) we decided to document it in the IPC Adapter project section (4.4).

Components, sprites and nodes of a scene are created by the static `SceneFactory` class and its loader interfaces. These are documented in detail in the section 4.2.

4.1.1 `Pin` class

`IsDriving` and `State` are the most important properties of the `Pin`. `State` is a read only getter property, that evaluates based on the state of the node it is connected to (if any). `IsDriving` can be changed by the owner `Component` by using either `SetHigh`, `SetLow` or `SetValue` methods. For read only pins, these methods trigger a warning, and the value isn't changed. If the value is different from before these methods were called, the *DrivingChangedEvent* is invoked. This notifies the connected node to update its internal driving pins counter.

By default, the pin is not connected to any node. `ConnectToNode` method is called within the `ElectricalNode` to change that. In this method, the pin is added to the node's list of pins. The `NotifyStateChanged` method is called by

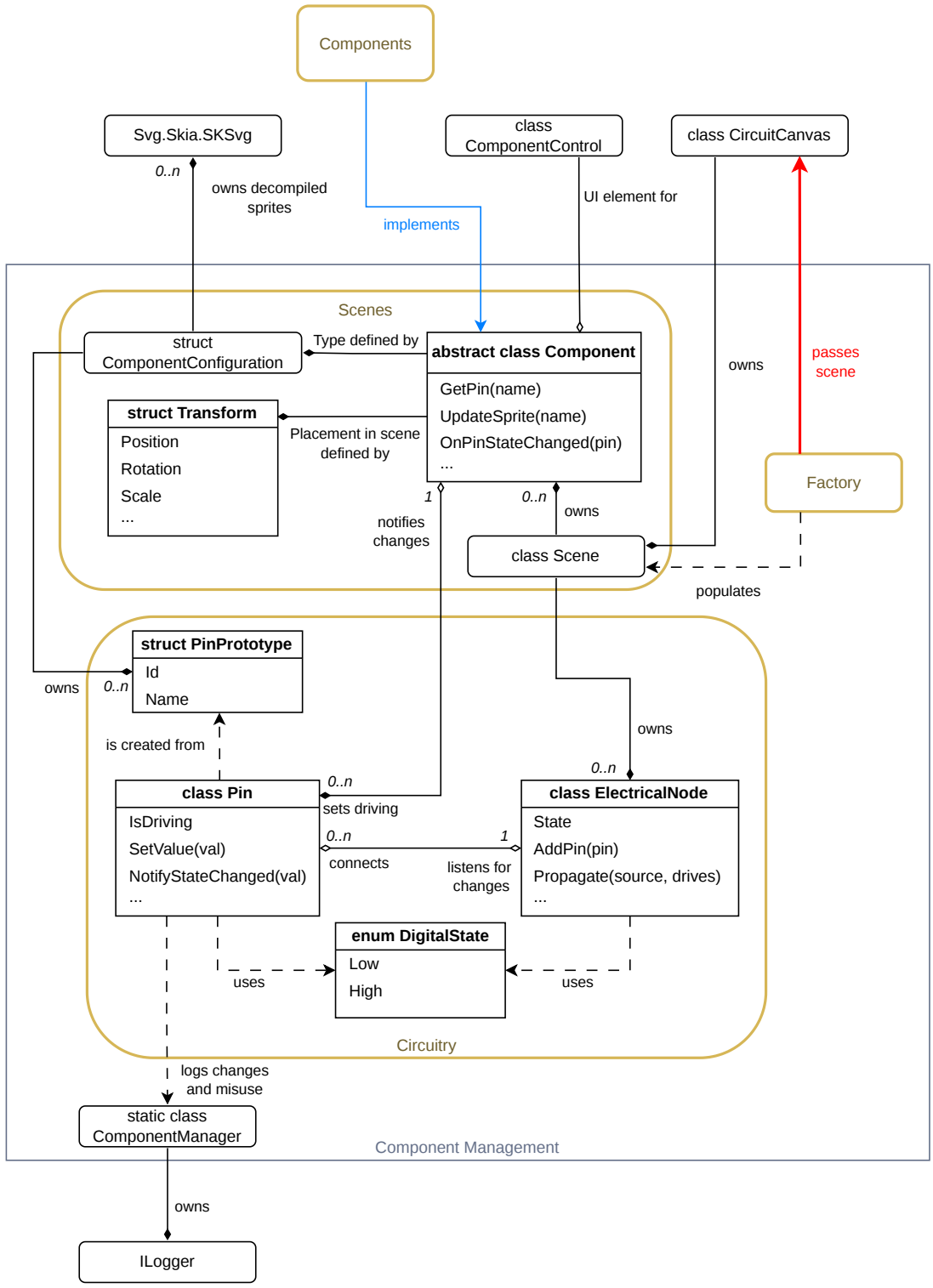

Figure 4.1 `Scenes` and `Circuitry` architecture of the `ComponentManagement` project

the node, when its state changes. The method is also called right after the pin has been successfully connected or disconnected.

The `Disconnect` method is implemented for cases, when `ConnectToNode` is called, while the pin is already connected to a different node. Or when such a request comes from the GUI. But at the time of writing this thesis, such mechanisms haven't been implemented yet. The pin is ever connected only once during the initialization of a node in the `SceneFactory`. These changes in connection are signaled by *PinConnectedEvent* and *PinDisconnectedEvent*.

If `NotifyStateChanged` is by any chance invoked on a write only pin, it triggers a warning, and the change isn't signalled to the component. The change is also not signalled when the pin is driving. In all other cases, pin's *StateChangedEvent* is invoked. This *event* is subscribed to by the parent `Component`, which handles it with its `OnPinStateChanged` virtual method.

The `PinPrototype` record struct is used to encapsulate the identifiers of pins on a given `Component` subtype. It holds only the `id` and `name` values and has no logic on its own. The prototypes are parsed from the component configuration file, and are held in the static `ComponentConfiguration` member of the `Component` subtype. They are passed to the `Pin`'s constructor during the completion of `Component` instances.

4.1.2 `ElectricalNode` class

The `ElectricalNode` holds a reference to all of its connected pins in a list. Its digital state is determined by how many of its connected pins are driving. It is low if this value is zero, high otherwise.

The pins of the node are modified using the `AddPin` and `RemovePin` methods. Inside these methods, the `ConnectToNode` and `Disconnect` methods are called on the pin passed in the argument. The `DrivingPins` count is updated. While the pin is connected, the `Propagate` method is either subscribed or unsubscribed from the pin's *StateChangedEvent*.

The `Propagate` method updates the `DrivingPins` counter. If the state has been changed by this update [1], the `NotifyStateChanged` is called on all pins connected to the node, except the one that triggered the change.

4.1.3 `Component` abstract class

An instance of the `Component` class is identified by both the name of the instance and the name of its type. It holds the `Transform` (position, rotation and scale within the scene), the currently rendered sprite, and instances of its pins. Reference to a pin is acquired through the `GetPin` methods, identified either by its name or id. The rendered sprite can be changed with the `UpdateSprite`, which also identifies sprite by its name, or its reference. These methods invoke the *SpriteChangedEvent*, which is listened to from the `ComponentControl`. This updates its visuals in the Avalonia UI window.

The user should define the logic of their custom type through the virtual methods. Not all of them need to be implemented, so they cannot be made abstract. The `OnControlPress` and `OnControlRelease` are called from the `ComponentControl`.

[1] I.e. when the counter changed from 0 to 1 or vice versa.

The `OnInitialized` is called from the `SceneFactory` once the component instance has been completed. `OnPinConnected`, `OnPinDisconnected` and `OnPinStateChanged` are subcribed to the appropriate *events* in their pin instances. The semantics and usage of these methods is better described in the subsection 5.2.3.

The `ComponentConfiguration` structs are stored statically inside the class as a dictionary, with component type name as the key. They store the metadata, but more importantly the pin prototypes from which pin instances are created, and the dictionary of compiled sprites, with the SVG file name as the key (excluding the `.svg` extension).

The pins are instantiated in the `InitPins` method, by passing the `PinPrototype` to the `Pin` constructor. Its *events* are subscribed to here as well.

Listing 4.1 `ComponentConfiguration.cs`

```
public record ComponentConfiguration() {
    public static readonly Vector2 DEFAULT_IMAGE_SIZE = new(50, 50);

    public required string Name;
    public required string ComponentPath;

    public string Description = "";
    public Vector2 ImageSize = DEFAULT_IMAGE_SIZE;
    public List<PinPrototype> Pins = [];
    public Dictionary<string, SKSvg> Sprites = [];
}
```

4.2 Component Management project - Scene factory

4.2.1 `SceneFactory` static class

The `SceneFactory` class ties the whole loading process together. It takes the path to the scene file and the directory containing the component assets in its constructor, saving them in private fields. It also holds the final `Scene` instance, which can be retrieved through a public property once the loading is complete.

The main work is done in the `LoadResources` method, which follows the flow shown in Figure 4.2. It starts by creating a new `ISceneLoader` to load the scene file, get the list of component types, and instantiate the raw components. It then loops through each component type found in the scene. For each type, it loads additional data from other files, and then configures every individual instance of that type. Finally, it instantiates the electrical nodes, packages them with the components into a new `Scene` object, and returns it.

The `CompleteComponentType` method handles the setup for a specific type of component. It first calls `CreateConfig` to get the `ComponentConfiguration` of the type. If that succeeds, it uses the `ISvgLoader` to load all SVG files from that directory. It resizes these SVGs to the size specified in the configuration and compiles them into usable sprites. These sprites are stored back in the configuration, and the finished configuration is registered globally by calling the component's static `AddConfiguration` method.

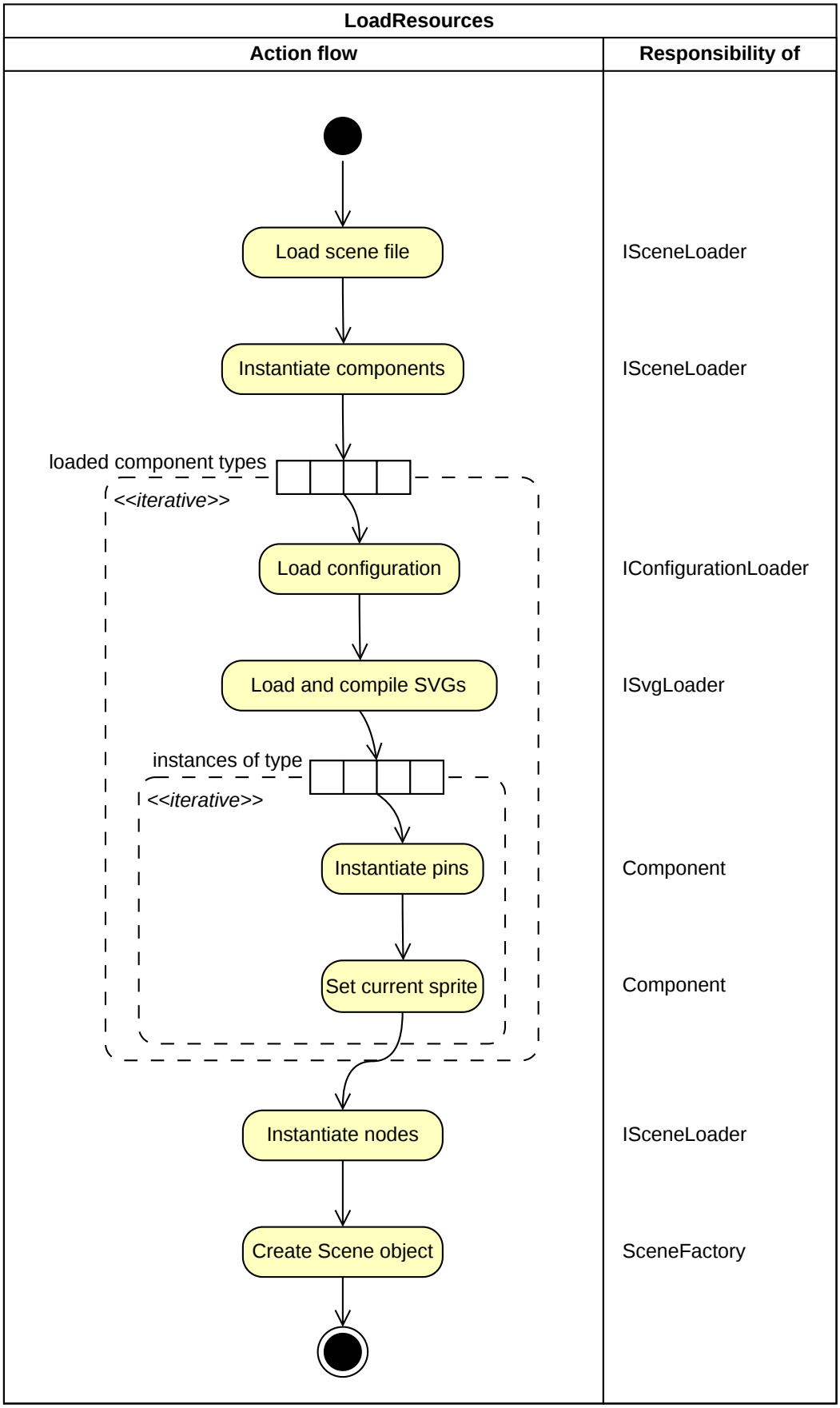

Figure 4.2 Action flow of the `SceneFactory.LoadResources` function

The `CreateConfig` helper method finds and loads the YAML configuration file for a component type. It looks inside the component's directory for a file with the exact same name as the directory, but with a `.yaml` extension. If the file exists, it uses the `YamlConfigurationLoader` to load and return the configuration. If the file is missing, it logs an error and returns null, which prevents the loader from trying to instantiate components of this type.

The `CompleteComponentInstance` method finalizes each individual component instance. It sets the base size of the component's SVGs to match the image size from its `ComponentConfiguration` and calls the internal pin initialization. It also assigns the first compiled sprite from the configuration as the active sprite, or fallback to an empty `SKSvg` if no sprites were loaded. At the end, it calls the component's `OnInitialized` method.

4.2.2 SVG Loader

`ISvgLoader` exposes functions for loading, modifying and compiling SVG documents. We say documents, because SVG is a readable XML-like format. So the first step is usually loading it into memory as a document object model. In this representation, we can freely change properties of individual picture elements are groups. We keep things simple by only changing the overall size of the picture to the default image size of a component type. After all necessary changes are done, the document is compiled into raw binary data, a blueprint on how the picture should be drawn. This data is wrapped an `SKSvg` type from `SkiaSharp`.

Listing 4.2 `ISvgLoader` interface

```
public interface ISvgLoader {
    public void LoadDocuments(string componentPath);
    public void ResizeDocuments(Vector2 desiredSize);
    public Dictionary<string, SvgDocument> GetDocuments();
    public Dictionary<string, SKSvg> CompileSvgs();
}
```

Parsing the file and compiling the document is very expensive. A scene with even a few changing SVGs had a considerable lag when using built-in Avalonia, or available Avalonia libraries. That's why we have to compile all the SVGs once before the scene is ready to be used. Multiple component instances can be drawn using the same image data, that's why we store it in the `ComponentConfiguration` struct, shared between all components of the same type. A single component only holds a reference to the image.

The `SkiaSvgLoader` uses the `SvgDocument` type from the `Svg` NuGet package to create the DOM from a file. On this type, the resizing is done using the `Width`, `Height` and `AspectRatio` properties. The compilation is done using `Svg.Skia` NuGet package, that extends both `Svg` and `SkiaSharp` libraries. The compilation is done using the static `CreateFromSvgDocument` function on `SKSvg` class. This is done for all files with the `.svg` extension in the component directory.

4.2.3 Configuration Loader

The `IConfigurationLoader` requires only one method, `LoadConfig`, that creates `ComponentConfiguration` struct from the given path. The implementation

`YamlConfigurationLoader` utilizes the `YamlDotNet` NuGet package. It implements a deserializer, that reads the YAML from a file and saved is it to memory as structure with equivalent fields and data types definitions. For that we define the `ConfigDto` struct [2].

Listing 4.3 `YamlConfigurationLoader.ConfigDto` struct

```
public record struct ConfigDto() {
    public required string Name;
    public string Description = "";
    public string ImageSize = "";
    public uint Pins = 0;
    public List<string> PinNames = [];
}
```

It serves as an intermediate step before we shape the loaded data into the form we need. The rest is quite simple. `Name` and `Description` have exactly the same. The `ImageSize` size arrived as a string with two numbers seperated by space, so we just split them and parse them separately. [3]The `Pins` are created as prototypes, where the unnamed prototypes have IDs ranging from zero to to the value specified in the DTO and no name. The named prototypes have names from the DTO and IDs in the same order as were they names listed in the DTO. The `Sprites` dictionary is not yet populated, as we want to avoid a dependency on another loader.

Listing 4.4 `YamlConfigurationLoader.CreateConfig` method

```
private ComponentConfiguration CreateConfig(
    ConfigDto configDto, string configPath) => new() {
        Name = configDto.Name,
        ComponentPath = Path.GetDirectoryName(configPath)!,
        Description = configDto.Description,
        ImageSize = YamlUtils.StringToVector2(configDto.ImageSize)
            ?? ComponentConfiguration.DEFAULT_IMAGE_SIZE,
        Pins = CreatePins(configDto)
    };
```

4.2.4 Scene Loader

The `ISceneLoader` is responsible for parsing the entire scene. That means creating both `Component` and `ElectricalNode` instances, and register all component types used in the scene. The component instances are not yet complete, their pins and sprite require data from the other loaders. That's why we need na effective way to group them by their type.

The `YamlSceneLoader` also uses the `YamlDotNet` package and its deserializer. The idea of the DTO also stays the same, just its definition is more complicated due to all the nested types. The `LoadFile` method does nothing more than deserializing the path into the DTO. The actual creation of the instances is done in `InstantiateComponents` and `InstantiateNodes`, which must be called in that order for the operation to make sense.

[2]The DTO meaning data transfer object.

[3]This is a non-standard YAML notation of 2D vectors. We created the static `YamlUtils` class to handle these vectors. See the specification at 5.2.1

Listing 4.5 `YamlSceneLoader.SceneDto` definitions

```
public class ComponentsMap : Dictionary<string, ComponentDto>;
public class TypesMap : Dictionary<string, ComponentsMap>;
public class NodesList : List<List<string>>;

public record SceneDto() {
    public required TypesMap Components { get; init; } = [];
    public required NodesList Nodes { get; init; } = [];
}

public record ComponentDto() {
    public string Position = "0 0";
    public float Rotation = 0;
    public string Scale = "1 1";
    public Dictionary<string, object>? ExtraProps { get; set; }
}
```

Instantiating components

`InstantiateComponents` loops through the nested dictionary of types and DTOs inside the deserialized scene. For each type name encountered, it attempts to resolve the C# type of the currently executing assembly (if not already cached) and then loops through individual component DTOs to kick off their creation.

The `CreateComponentInstance` method uses reflection to dynamically instantiate the object using its type name. Once the instance exists, it assigns it the information from `ComponentDto` (i.e. `Transform` and custom properties) and registers the finished instance into both the type-grouped and name-grouped lookup dictionaries.

The custom properties are processed in the `HandleExtraProps` method from the `ExtraProps` member of the `ComponentDto`. User-defined properties are not part of the base component layout. They are acquired through reflection by matching properties of the concrete component class against the dictionary keys. If a property exists, the input type is converted dynamically to the destination property type, and the value is updated.

Listing 4.6 `YamlSceneLoader.CreateComponentInstance` method

```
private void CreateComponentInstance(
    string typeName,
    string componentName,
    ComponentDto componentDto)
{
    try {
        Type type = _typesByName[typeName];
        Component compInstance =
            (Activator.CreateInstance(type, typeName)
                as Component)!;

        compInstance.Name = componentName;
        compInstance.Transform = ParseTransform(componentDto);
        HandleExtraProps(compInstance, componentDto.ExtraProps);

        _instancesByType[typeName].Add(compInstance);
        _instancesByName[componentName] = compInstance;
        ComponentManager.LogInfo("...");
```

```
19      }
20      catch {
21          ComponentManager.LogError("...");
22      }
23  }
```

Instantiating nodes

After all components exist, the `InstantiateNodes` iterates through the list of node arrays, creating a new `ElectricalNode` container for each. It splits the string representations (e.g. "Component.Pin") to locate the parent instance by name, resolves the pin using either its numeric index or name identifier, and adds `Pin` references to the node.

4.3 Main project with Avalonia UI

The `Main` project has been imlemented with the MVVM architecture in mind. The only model of the application is the `Component` class from the `Component Management` project (see 4.1).

The views and viewmodel files are put in separate directories. Views consist of `.axaml` files and the codebehind `.axaml.cs` files. These codebehind files contain a partial class with a name of the the view, inheriting from Avalonia's `UserControl` class. These can be used to write logic for the views as well, but we deliberately left them empty. The only exception is the `App.axaml.cs`, which has to call the `MainController` once the UI is initialized.

The `.axaml` files define the visual tree of the application. The root of this application is `App.axaml`, which owns the main window and includes some default styles from `Styles/General.axaml`. Our application doesn't create any other windows, everything is situated in the `MainWindow.axaml` and its controls.

The `MainController` intializes the majority of the application. It invokes the `SceneFactory` and populates the `CircuitCanvas` viewmodel with the parsed scene. Then it chooses the IPC implementation based on the command line arguments, and the passes the IPC instance to the `Arduino` component.

4.3.1 `Program.cs`, `App.axaml` and `MainWindow.axaml`

The project has been created from an Avalonia UI template, which includes these files described in this subsection. The entry point file `Program.cs` with the `Main` function has remained unchanged. It only builds the Avalonia application using `App` class and nothing else.

The `App` class is defined by the `App.axaml` and `App.axaml.cs` files. The `.axaml` consists of nothing else but the `<Application.Style>` element, which includes styles from the `Styles/General.axaml`. It mainly defines background colors for panels and canvases.

Much more important is the `App` codebehind. It overrides the method `OnFrameworkInitializationCompleted` inherited from the Avalonia's `Application` class. At this point we can access to the `ApplicationLifetime`. It is an interface that defines in what kind of environment our application runs,

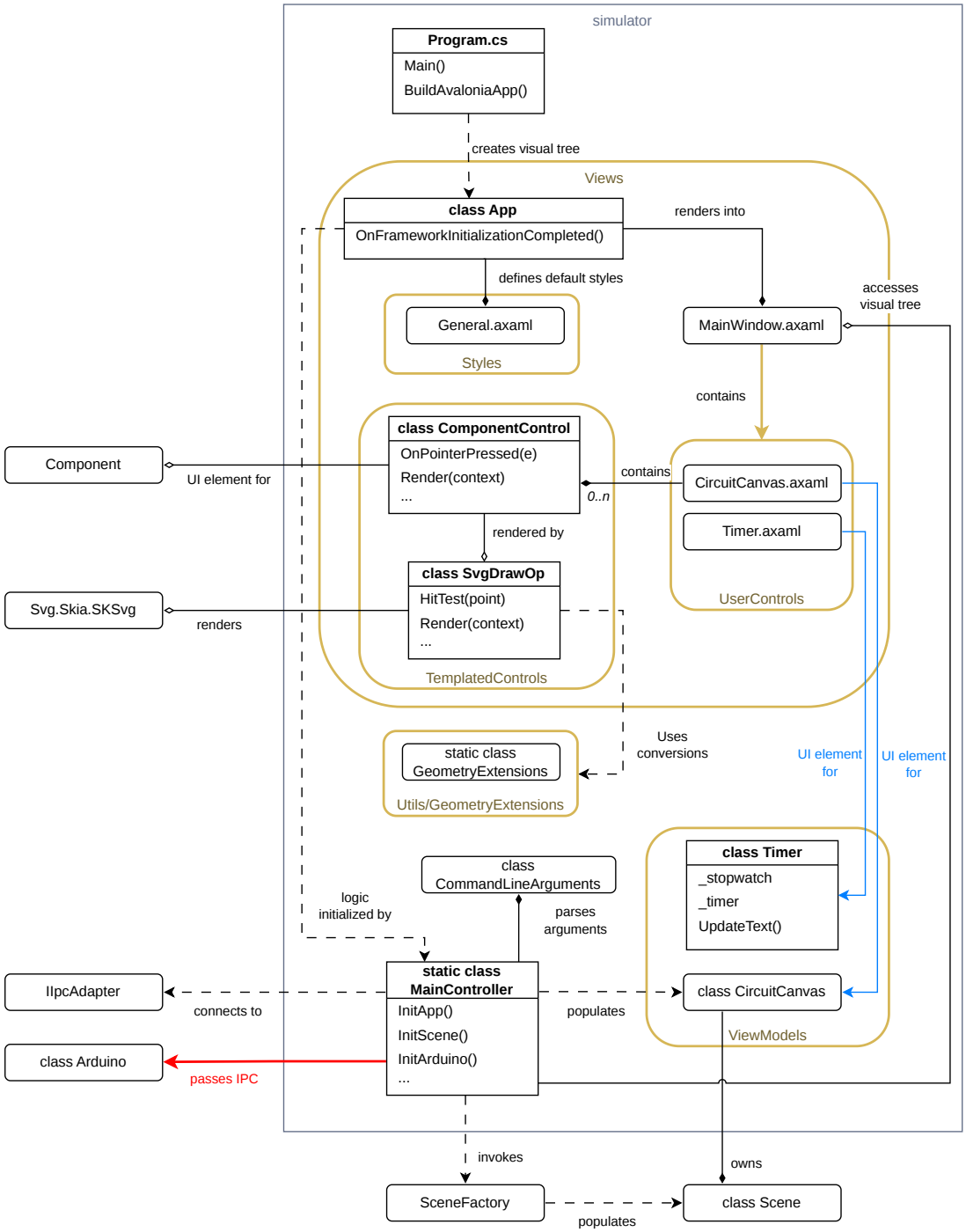

Figure 4.3 `Main` project architecture

like if we can have one or multiple windows, or when should the application be shut down. This is an important factor when we develop for Android or iOS for example. We target Windows and Linux PCs, so our lifetime is of type `IClassicDesktopStyleApplicationLifetime`. We create the instance of `MainWindow` class and pass it to the lifetime. Then we call our `MainController` with the lifetime as an argument.

Listing 4.7 `App.OnFrameworkInitializationCompleted` override

```
public override void OnFrameworkInitializationCompleted() {
    MainWindow mainWindow = new MainWindow();

    if (ApplicationLifetime is
        IClassicDesktopStyleApplicationLifetime desktop)
    {
        desktop.MainWindow = mainWindow;
        MainController.InitApp(desktop);
    } else {
        throw new NotImplementedException("...");
    }

    base.OnFrameworkInitializationCompleted();
}
```

Main window

Layouts of views (more accurately controls) are defined by so-called panels. Panels are designed to contain other controls in an organized manner. The layout of the `MainWindow` uses the `Grid` panel. It works very similarly to how the grid display method works in CSS. The cells of the grid are merged into bigger wholes, which contain other panels. Between these wholes are placed elements called `GridSplitter`, that can resize the wholes by dragging them with a cursor.

The grid has been prepared such that menu bar and toolbar would be at the top, inspector of components would be on the left, the circuit canvas on the right, and the log messages would be displayed at the bottom. Out of these, only the circuit canvas has yet been implemented.

The timer visible in the main window simply shows the time elapse from the start of the frontend application. It is implemented by the user control `Timer` consisting of a single text block. Its viewmodel utilizes the .NET's `Stopwatch` and Avalonia's `DispatcherTimer` objects, to periodically update the text. In future this view should be used to display the simulation time retrieved from backend.

4.3.2 `MainController` static class

This class initializes all the important systems and brings them together. It stores a reference to the `MainWindow` from the desktop lifetime, so that we can access the visual tree. Then it parses and stores the program options with the `CommandLineArguments` struct, implemented with the `CommandLineParser` NuGet package. After that it initializes loggers, the scene, the IPC and the `Arduino` component, if present in the scene.

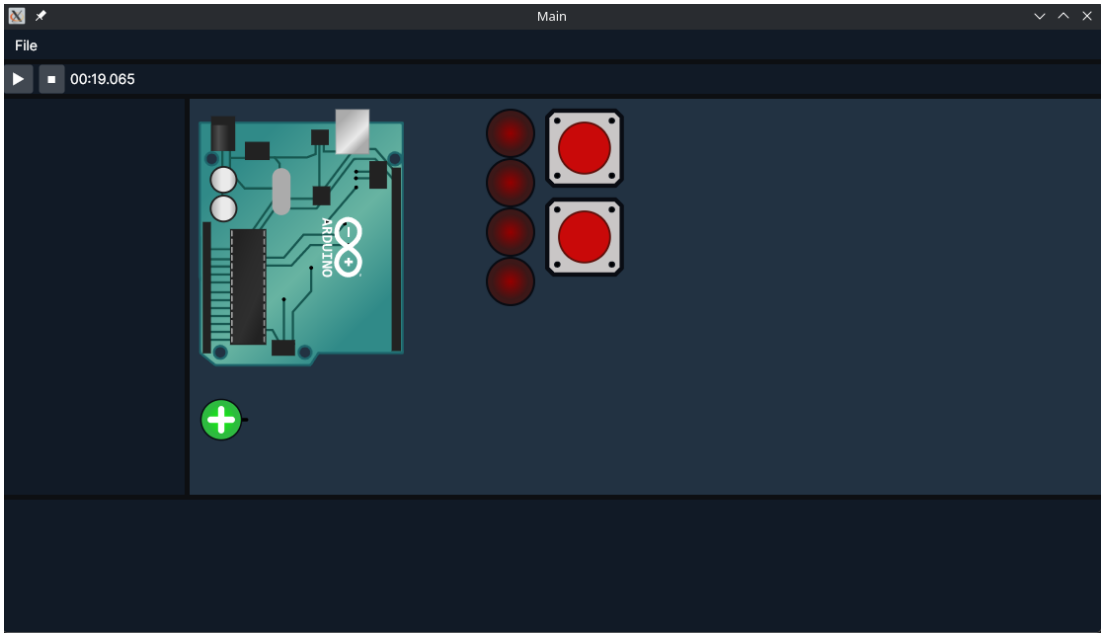

Figure 4.4 The `MainWindow` layout

Scene initialization

Listing 4.8 `MainController.InitScene` function

```
private static void InitScene() {
    var SceneFactory = new SceneFactory(
        Arguments.ScenePath, Arguments.ComponentsPath);
    Scene = SceneFactory.LoadResources();

    var canvas = new CircuitCanvas();
    foreach (var comp in Scene.ComponentsMap.Values) {
        canvas.Components.Add(comp);
    }

    MainWindow.CircuitCanvas.DataContext = canvas;
}
```

The scene is created by the `SceneFactory` from the files specified in the command line argument. Then the `CircuitCanvas` viewmodel is created, and its `Components` property is populated with the components from the scene. Then the viewmodel is set to be the data context for the `CircuitCanvas` view. But what does setting a data context mean?

As we mentioned at the beginning of this section, all the views have a codebehind with a partial class named after the view. The other parts of this class are generated during compilation of the `.axaml` file. The XAML knows the name of the partial class it should generate thanks to the mandatory `x:Class` attribute in the root element of the control.

We never explicitly defined a `CircuitCanvas` property in the `MainWindow` codebehind. Instead, we added the attribute `x:Name="CircuitCanvas"` to the concrete `CircuitCanvas` XAML element. This tells the XAML compiler to generate the property with the given name in the `MainWindow` class, and populate it automatically with the concrete instance during Avalonia initialization. That is

with all the properties set either through the codebehind, or through the attributes on the XAML element itself. [11]

`DataContext` is a property inherent to all the controls. It can be an instance of any one class, with which the control exchanges data through binding. That means, the control can change properties automatically when the values of the bound property changes. And in the other direction, the property of the data context automatically updates when a user interacts with the control. [4]

In MVVM, data contexts are the best way to bind views to their viewmodels. [12] The type of the `DataContext` must be set to match the type of the view model in XAML. This can be done in more than one way. We do it by adding the `x:DataType` attribute to the root element of `CircuitCanvas.axaml`. The code below shows a simplified XAML example of how these two steps have been done.

Listing 4.9 Data context setup in XAML

```
<Window ...
        xmlns:uc="clr-namespace:Gui.Views.UserControls"
        x:Class="Gui.Views.MainWindow"
        Title="Main">

    <uc:CircuitCanvas x:Name="CircuitCanvas"/>

</Window>

<UserControl ...
             x:Class="Gui.Views.UserControls.CircuitCanvas"
             x:DataType="Gui.ViewModels.CircuitCanvas">

    <ItemsControl ItemsSource="{Binding Components}">
        ...
    </ItemsControl>

</UserControl>
```

Arduino initialization

First we try to find the `Arduino` component in the scene. It is perfectly valid to have a scene without Arduino, so in that case we only throw a warning and skip the rest of the initialization. With the way this is handled, only one Arduino can reasonably work in a scene. A better implementation, would be to initialize the Arduino with IPC inside the component itself, and get the IPC arguments like port or shared memory name from extra properties in the scene file. That way, it would be possible to have multiple Arduino components in the scene, each connected to its own simulator. But the approach has not been properly tested. Since the IPC arguments are at the moment passed through the command line, we can only have one Arduino anyway.

Subsequently, the IPC logger, the text encoder and the IPC adapter is initialized. The `IIpcAdapter` implementation is chosen based on the `IpcType` option

[4] That is if the properties of the data context class are set properly. For example by implementing the `INotifyPropertyChanged` interface or using the attributes from `CommunityToolkit.Mvvm` NuGet package [13]. In our case with `CircuitCanvas`, we used the `ObservableCollection` class from the same package, to store the `Component` instances in.

from the command line arguments. If the chosen type is `None`, the function is skipped completely. After instantiating the adapter, we perodically try to connect to it in a new task, and immediately pass it to the `Arduino` component. Once the adapter connects (if ever), the component will be able to react to the incoming events.

4.3.3 `ComponentControl` custom control

The `ComponentControl` wires up the elements of the circuit canvas to the `Component` class in the `Component Management` project. The class inherites from `Control`, in order to override its `Render` method, and respond to Avalonia *events*. The `InvalidateVisual` method is called when the sprite of the `Component` changes. It notifies the Avalonia renderer, that it is time to redraw the control. The `Render` method is then called, using the new sprite.

`OnPointerPressed` and `OnPointerReleased` are tied to the components `OnControlPress` and `OnControlRelease` methods. These are called when the user clicks the control on the canvas with their mouse. The Alt key modifies whether should the control be still pressed, when the pointer is released.

We need to specify which `Component` instance from `CircuitCanvas` view is tied to this control. I.e. we need to define a property, we can set up from the canvas' XAML, that we can then access from the control. Unfortunately, Avalonia is very obtuse with its property system, and requires quite a lot of boilerplate.

First we define the `_component` backing field. Then we define a static `DirectProperty` field, initialized by the `RegisterDirect` method, templated by the type of the control and type of the backing field. Then we define a regular property with getter and setter. The setter must contain the `SetAndRaise` in order to notify the control when its value changes. The property is passed to `RegisterDirect` and `DirectProperty` is passed to `SetAndRaise`. Then, once the value of the property changes through XAML binding, the `OnPropertyChanged` method is called. From the argument of this call we determine the property changed was our `Component`, and we can subscribe or unsubscribe from its *SpriteChangedEvent*.

Listing 4.10 Property setup in `ComponentControl`

```
1  private Component _component = null!;
2
3  public static readonly DirectProperty<
4      ComponentControl, Component> ComponentProperty =
5          AvaloniaProperty
6          .RegisterDirect<ComponentControl, Component>
7              (nameof(Component), o => o.Component,
8                  (o, v) => o.Component = v);
9
10 public Component Component {
11     get => _component;
12     set => SetAndRaise(ComponentProperty,
13         ref _component, value);
14 }
15
16 protected override void OnPropertyChanged(
17     AvaloniaPropertyChangedEventArgs change) {
18         base.OnPropertyChanged(change);
```

```
19          if (change.Property == ComponentProperty) {
20              HandleComponentChanged(change);
21          }
22  }
```

We used direct properties since they are the most performant and appropriate for read only values (we don't change the `Component` from the control itself). There are more types of properties appropriate for different use cases. [14]

4.3.4 SvgDrawOp drawing operation

The `SvgDrawOp` is used within the `Render` method of `ComponentControl`. It takes the compiled SVG of the `Component` as an argument, as well as its `Transform` converted into a transformation matrix. This conversion is done through an extension method within the `GeometryExtensions` class. The method creates elementary 3x3 translation, rotation and scale matrices and multiplies them together. `SKMatrix`, the type of the transformation matrix, comes from the `SkiaSharp` library. The same one that defines the SVG type, and many other systems used by the Avalonia's rendering pipeline. [5]

Hit testing

Listing 4.11 `SvgDrawOp.HitTest` method

```
1  public bool HitTest(Point p) {
2      SKPoint point = p.ToSKPoint();
3      SKPoint localSpacePoint =
4          Transform.Invert().MapPoint(point);
5      return _hitBox.Contains(localSpacePoint);
6  }
```

This method is called during pointer *events*, e.g. when the user clicks on the canvas. It checks whether is the pointer located on top of the control. We use this in order to trigger the `OnPointerPressed` and `OnPointerReleased` handlers of the `ComponentControl`.

The argument `p` is the location of the pointer relative to the circuit canvas. The `_hitBox` is given by the SVG. It is the rectangle in which the picure is inscribed. We described its coordinates in the local space of the picture. The rectangle's left corner is located in the origin of the local space, and its dimensions are given by `ImageSize` property we specified in the component's configuration. Using the inverse of the tranformation matrix, we can map `p` from the space of the canvas, to the picture's local space. If it lies within the `_hitBox` area, we consider it a hit.

Rendering

Listing 4.12 `SvgDrawOp.Render` method

```
1  public void Render(ImmediateDrawingContext context) {
2      var leaseFeature = context.TryGetFeature<
```

[5] `SkiaSharp` is the C# API of `Skia`, an open source 2D graphics engine. It is owned by Google and is used in some of their products like Chrome or Android for rendering PDFs, shaders and vector graphics. [15] The API is implemented in many more popular languages.

```
        ISkiaSharpApiLeaseFeature>();

    if (leaseFeature == null) {
        throw new Exception("LEASE");
    }

    using var lease = leaseFeature.Lease();
    var canvas = lease.SkCanvas;
    canvas.Save();

    canvas.Concat(ref Transform);
    canvas.DrawPicture(Svg.Picture);
    canvas.Restore();
}
```

The method takes `ImmediateDrawingContext` as an argument, which is passed down from the argument of the `Render` method of `Control`. It defines the environment in which the picture is going to be drawn. We need to access the API of the Skia's canvas, upon which the Avalonia's drawing contexts are built and render their own vector graphics. We use it to draw our own `SKSvg` sprite. We do that by obtaining the SkiaSharp API lease from the context's features. [6]

The Skia canvas coordinates are already in the vector space of the parent control (the `CircuitCanvas`). So we can just multiply the transformation matrix of the canvas with the transformation matrix of the control, in order to place it on the canvas with the right translation, rotation and scale. The `Concat` method does exactly that. Then we rasterize the picture with the `DrawPicture` method.

We need to call `Save` and `Restore` methods, because the same canvas will be used for many more controls on the visual tree, we can't leave it in the local space of our control.

About `Save`: "This call saves the current matrix, clip, and draw filter, and pushes a copy onto a private stack. Subsequent calls to translate, scale, rotate, skew, concatenate or clipping path or drawing filter all operate on this copy. When the balancing call to Restore() is made, the previous matrix, clipping, and drawing filters are restored." [16]

4.3.5 `CircuitCanvas` user control

Listing 4.13 `CircuitCanvas` view layout

```
    <Panel ClipToBounds="False">
        <ZoomBorder Stretch="Fill"
                    ClipToBounds="True"
                    EnableDoubleClickZoom="False">

            <ItemsControl ItemsSource="{Binding Components}">
                <ItemsControl.ItemsPanel>
                    <ItemsPanelTemplate>
                        <Canvas ClipToBounds="False"/>
                    </ItemsPanelTemplate>
                </ItemsControl.ItemsPanel>
```

[6] The notion of features and leases is specific to Avalonia, and at the time of writing the thesis doesn't seem to be properly documented. The usage has been inspired by the Avalonia's sample projects [17]

```
12
13                  <ItemsControl.ItemTemplate>
14                      <DataTemplate DataType="scenes:Component">
15                          <tcontrol:ComponentControl
16                              Component="{Binding}"/>
17                      </DataTemplate>
18                  </ItemsControl.ItemTemplate>
19              </ItemsControl>
20
21          </ZoomBorder>
22      </Panel>
```

The `CircuitCanvas` is a user control, that contains component controls as its children. Those are stored in a collection, in the data context (viewmodel) of the `CircuitCanvas`. How the items of this collection are laid out is defined by the `ItemsControl` element. It inserts certain elements into the visual tree for each of the items in the collection. It binds to the collection, and exposes two properties: `ItemsPanel` and `ItemTemplate`.

`ItemsPanel` describes the layout of the container in which the items will be drawn. We want to freely choose the exact coordinates of where our components should be drawn to. So we chose to use `Canvas`, a type of panel made exactly for that purpose. `ItemTemplate` describes how should the individual items be styled. This could be a text box, another panel, or our custom drawn `ComponentControl`. The control must be enclosed in a `DataTemplate` element. The data type of the collection's item is specified through its `DataType` property. Using the `Binding` keyword, we assign each of the collection's elements to one of the controls.

The `ZoomBorder` element from the `PanAndZoom` NuGet package allows us to freely pan and zoom its child element using the mouse buttons. Unfortunately this moves with the entire `Canvas` element. And because of how most panels are implemented, children located outside its bounds are never rendered. I.e. any elements with at least one negative coordinate, and elements with coordinates greater than the size of the canvas in its unzoomed state. The canvas fills the entirety of its parent's space, so more elements can fit into it by resizing the panel, but of course that's not a good solution. We made numerous unsuccessful attempts to fix the canvas in one place, and instead concatenate the transformations of components and the transformation of the `ZoomBorder`. We didn't find a way to bind to the border's events.

The viewmodel of the canvas itself is very simple. It contains just one property of the `ObservableCollection` type from the `CommunityToolkit.Mvvm` package. It ensures that every time an item is added, removed or modified within the collection, the properties binding to this collection are properly notified.

Listing 4.14 `CircuitCanvas` viewmodel

```
1 public class CircuitCanvas : ObservableObject {
2     public ObservableCollection<Component>
3         Components { get; } = new();
4 }
```

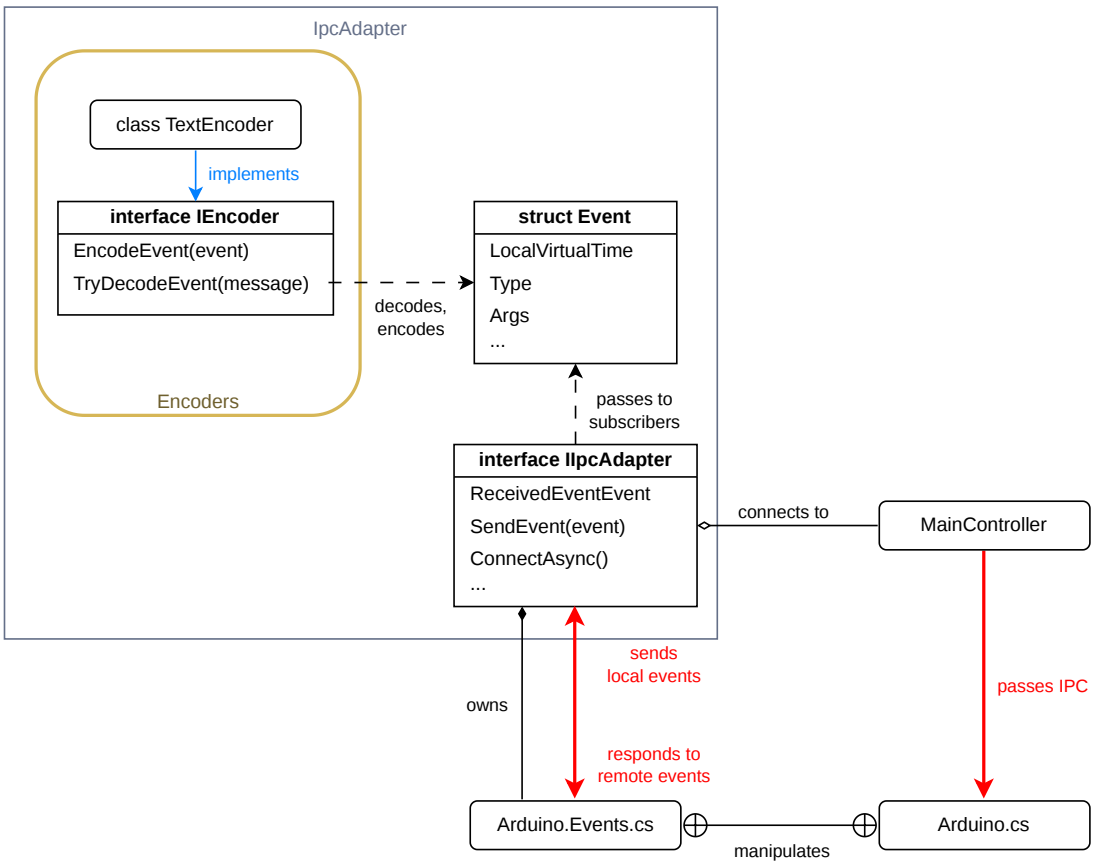

Figure 4.5 `IpcAdapter` project architecture

4.4 IPC Adapter project

This project attempts to unify the behavior of the TCP adapter and the shared memory adapter in a similar manner the `ipcAdapter.hpp` did in the backend. `IIpcAdapter` is initialized by the `MainController` and the implementation is chosen based on command line arguments. It is then passed to the `Arduino` component to handle.

This time around, the adapter is not dependent on a `Simulator`-like entity to pass the events to. It simply invokes the *ReceivedEventEvent*, which passes the event to any subscribed handler, i.e. the `Arduino` class. The actions of the `Event` struct are also not defined by static methods, but determined in the `Arduino` class itself.

4.4.1 `Event` struct

Listing 4.15 `Event` struct

```
1 public enum EventType { Unknown, Write, PinMode, Reboot }
2
3 public struct Event {
4     public const int MAX_ARGS = 3;
5
6     public EventType Type;
7     public long TimestampMicros;
8     public long LocalVirtualTime;
```

```
    public int[] Args = new int[MAX_ARGS];

    public Event() { }
}
```

We've seen this structure many times before. In contrast with its backend counterpart, it doesn't include the `action` lambda, the factory methods, nor any other helper functions.

4.4.2 `IIpcAdapter` interface

Listing 4.16 `IIpcAdapter` interface

```
public interface IIpcAdapter {
    private const int RECONNECT_DELAY = 3000;
    public bool IsConnected { get; }
    public event Action<Event> ReceivedEventEvent;

    public void SendEvent(Event @event);

    public Task<bool> TryConnectAsync();
    public void Disconnect();

    public async Task ConnectAsync() {
        while (!await TryConnectAsync()) {
            await Task.Delay(RECONNECT_DELAY);
        }
    }

    public void Reconnect() {
        Disconnect();
        Task.Run(ConnectAsync);
    }
}
```

The `SendEvent` method is similar to the `on_event` method of the `ipcAdapter.hpp`. It is also used to send an event over IPC, by first encoding it to a message, just the dependencies are now inversed. The adapter isn't dependant on the simulator but vice versa. So we don't need to pass it anywhere, just call it directly from a component. The events are not directly applied by calling the component, but by invoking the *ReceivedEventEvent* that the components subscribe to.

The interface also offers some more methods for handling the connection. `TryConnectAsync` and `Disconnect` must be implemented by the concrete adapters as they are dependant on the technology used. We want them exposed, because in the future, it could be desirable to change the IPC setup during runtime (for example, connecting to a simulator on a different port).

`ConnectAsync` tries to periodacally connect to the every three seconds. Useful when the simulator isn't running yet. It establishes connection automatically when it is ready. `Reconnect` is useful, when there was some kind of sudden failure.

4.4.3 `TextEncoder` class

The `IEncoder` and `TextEncoder` are nearly equivalent to `encoder.hpp` and `textEncoder.cpp` in the backend. The interface still exposes methods for encoding and decoding, where decoding return a success state, and outputs the event to a parameter.

The implementaion defines constants for formats of individual parts, and it still uses string formatting to fill them with event's arguments. Just the decoding has become much more convenient thanks to .NET's built-in functions.

4.4.4 Arduino component

The configuration of the component defines 14 digital pins (`D0` up to `D13`) and 6 analog pins (`A0` up to `A5`). It has exactly one sprite, simply named `Arduino`. The partial class `Arduino` is split into two files `Arduino.cs` and `Arduino.Events.cs`. This is to establish separation of concerns while still having access to all the private methods of both the side handling pin interaction and the side handling consumption and creation of events.

`Arduino.cs`

The front end side of the class defines a method for connecting to the IPC and some helper functions for conversion between pin indices and pin instances. The rest of the methods each reflects one of the possible IPC events. `WriteToPin` sets a value on a digital pin,`SetPinMode` sets a pin to write only or read-only mode, and `Reboot` defaults all pins to a low state.

Listing 4.17 `Arduino.ConnectToSimulator` method

```
public void ConnectToSimulator(IIpcAdapter ipc) {
    _ipc = ipc;

    _ipc.ReceivedEventEvent += e => {
        _uiContext.Post(_ => ProcessRemoteEvent(e), null);
    };
}
```

The class gets the `IIpcAdapter` object from the `MainController` class of the `Main` project. It is already connected to the IPC, or trying to connect periodically. We have to subscribe to its *ReceivedEventEvent*, which passes the decoded event as an argument.

We have to be careful about the thread from which we consume the event. Most UI frameworks are single-threaded, and don't handle well, if more than one thread tries to enter the rendering pipeline. Consuming an event means we will have to at some point change a sprite of component connected to Arduino, and thus invalidate a visual of its associated `ComponentControl` in an Avalonia view. But the IPC listening system most likely executes on a different thread than the UI. Because of this the application kept freezing almost immediately after launch of the IPC, probably due to some deadlock.

We solved it by wrapping the `ProcessRemoteEvent` in `_uiContext.Post` method. `_uiContext` is an instance of the .NET's `SynchronizationContext` class. It represents the execution context of a single thread. We set the instance

by calling `SynchronizationContext.Current` while constructing the component in the UI thread (the one that executes `MainController` functions). The `Post` method is a non-blocking method that schedules a delegate to be executed on the given thread. So the *ReceivedEventEvent* is subscribed to a lambda, which schedules the consumption of the event on the correct thread.

Listing 4.18 `Arduino.OnPinStateChanged` method

```
public override void OnPinStateChanged(Pin pin) {
    int idx = GetPinIndex(pin.Name!);

    if (_ipc?.IsConnected ?? false) {
        InvokeWriteEvent(idx, pin.State);
    }
}
```

The rest of the logic is simple. If we detect a change on one of the input pins, we check that IPC is connected. If it is, we create the appropriate event with the appropriate arguments.

`Arduino.Events.cs`

The `ProcessRemoteEvent` method comes from the event side of the `Arduino` class. It calls one of the `WriteToPin`, `SetPinMode` or `Reboot` methods based on the type of the event. Then it stores the LVT of the event into the `_lastSeenLvt` member.

Listing 4.19 Sending the Write event with `Arduino.Events.cs`

```
private Event CreateWriteEvent(int pinId, DigitalState value) =>
    new() {
        Type = EventType::Write,
        Args = [ pinId, (int)value ]
    };

private void InvokeWriteEvent(int pinId, DigitalState value) =>
    SendLocalEvent(CreateWriteEvent(pinId, value));

private void SendLocalEvent(Event @event) {
    @event.LocalVirtualTime = _lastSeenLvt;
    _ipc?.SendEvent(@event);
}
```

When the component needs to produce an event, we call one of the `Invoke*` methods. It creates the `Event` struct with the specific type and arguments and sends it via IPC with the `_lastSeenLvt`. It isn't necessary to assign a unique LVT to different events. They will always be sent to the backend in the same order. All that matters is that a certain set of local events occurred between 2 remote events produced by the backend.

4.5 TCP Adapter project

4.5.1 `TcpClient` class

The TCP Adapter is built on the .NET's native TCP Client implementation. It uses the `NetworkStream`, through which the actual data is sent. Our own

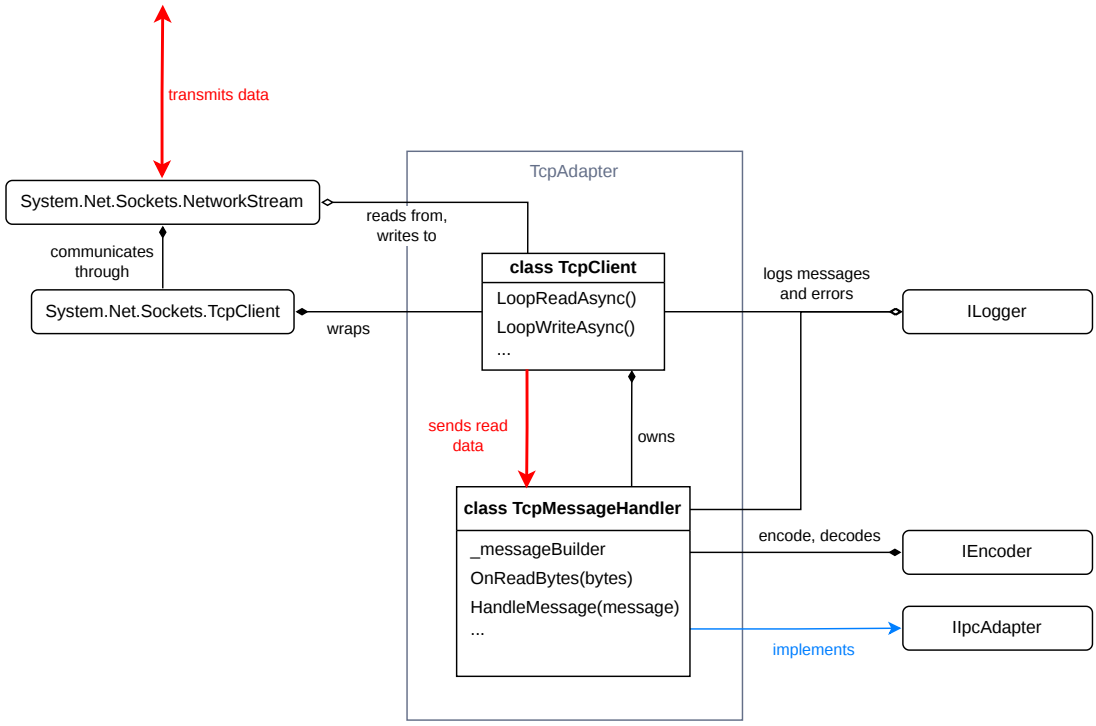

Figure 4.6 `tcpAdapter` project architecture

`TcpClient` class wraps around the client. It establishes connection on the given port and initializes loops for sending and receiving data in new tasks. The received chunks of data are passed to the `TcpMessageHandler`, which divides them into individual messages. Messages to be sent are passed over to the `TcpClient` through the handler as well. `TcpMessageHandler` implements the `IIpcAdapter`.

The `TcpClient` is constructed with an IP address and a port (i.e. localhost and the port of the backend). Only in the `TryConnectAsync` method is the underlying `_client` created and connected to asynchronously. Its `NetworkStream` is acquired. The `LoopReadAsync` and `LoopWriteAsync` methods are then run in separate tasks. The cancellation token (`_cts`) is created. The token can be used to terminate these loops, once a disconnect or an error occurs. The `Disconnect` method cancels this token, and disposes of both the stream and the client.

Receiving messages

Listing 4.20 `TcpClient.LoopReadAsync` method

```
1  private async Task LoopReadAsync() {
2      try {
3          while (!_cts.IsCancellationRequested) {
4              int bytesRead = await _stream!.ReadAsync(
5                  _buffer, 0, _buffer.Length, _cts.Token);
6              if (bytesRead == 0) {
7                  ReachedEof?.Invoke();
8                  return;
9              }
10
11             _logger?.LogDebug("...");
12
```

```
                var byteChunk = Encoding.ASCII.GetString(
                    _buffer, 0, bytesRead);
                ReadBytesEvent?.Invoke(byteChunk);
            }
        }
        catch (Exception e) {
            _logger?.LogError("...");
        }
    }
```

The read loop is conditioned by the `_cts` token. Inside, the stream's `ReadAsync` method is used to read any new available data on the stream, and copy it to the `_buffer`. `_buffer` is a simple array with size of 4096 bytes. We could choose any other size, but this one gives us a lot of breathing room if the stream happens to get clogged by many messages. The method blocks until new data are available.

By passing 0 to the 2nd argument of the method, new data will always be copied to the start of the buffer, overwriting any old data. By passing the `_cts` token as argument, the method will throw an `OperationCancelledException` once the token has been canceled. The method returns the number of bytes it has successfully read. If this number is 0, it means the connection has been gracefully closed. [18]. In that case we invoke the *ReachedEof event*.

Since the new data is always at the start of the buffer, we can pass it to the `Encoding.ASCII.GetString` function in order to convert it into a string of ASCII characters. The number of bytes we want to convert is the value `bytesRead` returned from `ReadAsync`. We pass the converted bytes to the *ReadBytesEvent*, to which the `TcpMessageHandler` subscribes to. There the string can (but doesn't have to) be pieced together into a list of meaningful messages.

Sending messages

Listing 4.21 `TcpClient.SendMessageAsync` method

```
    public async Task SendMessageAsync(string message) {
        await _sendQueue.Writer.WriteAsync(message);
    }
```

For sending messages we utilize the .NET's `Channel` class. This is a thread safe implementation of the concurrent producer/consumer pattern. It is a FIFO data structure, so we call its instance `_sendQueue` in our code. The messages are added to the channel through the `Writer` property and removed through the `Reader` property. The channel is unbounded, meaning that it doesn't block the writing thread, if the number of items reaches a certain limit.

Listing 4.22 `TcpClient.LoopWriteAsync` method

```
private async Task LoopWriteAsync() {
    try {
        while (await _sendQueue.Reader.WaitToReadAsync()) {
            while (_sendQueue.Reader.TryRead(out var message)) {
                var data = Encoding::ASCII.GetBytes(message);
                await _stream!.WriteAsync(data, 0, data.Length);
                _logger?.LogDebug("Sent {data.Length} bytes...");
            }
        }
    }
```

```
    catch (Exception e) {
        _logger?.LogError("TCP Writing loop failed: {e.Message}");
    }
}
```

The write loop reads from the `_sendQueue`. It blocks the thread until there is at least one item available to be read. If the channel has been invalidated, the method returns false. In the inner loop, all available string messages are read and converted to ASCII bytes. The bytes are then copied asynchronously into the network stream.

4.5.2 `TcpMessageHandler` class

The `TcpMessageHandler` implements the `IIpcAdapter` and owns our `TcpClient` wrapper. It let's the client handle the `TryConnect` and `Disconnect` interface method by calling the client's methods with the same name. The `SendEvent` method is implemented by encoding the event, appending the delimiter, and passing it to the client's `SendMessageAsync` method in a new task. When the client's *ReachedEof* is invoked, the handler tries to reconnect using the method inherited from the interface.

Handling incoming messages

Listing 4.23 `TcpClient.OnBytesRead` method

```
private void OnBytesRead(string bytechunk) {
    foreach (char c in bytechunk) {
        if (c == Delimiter) {
            HandleMessage(_messageBuilder.ToString());
            _messageBuilder.Clear();
        }
        else {
            _messageBuilder.Append(c);
        }
    }
}
```

The handler's `OnBytesRead` method subscribes to the client's *ReadBytesEvent*. It receives a string that may contain the entire message, multiple messages, or just their pieces. So we add any incoming characters to the `_messageBuilder` and look for the delimiter. Once a whole message has been put together, it is passed to the `HandleMessage` method, and the builder is cleared.

In `HandleMessage`, the message is decoded into an `Event` struct and the *ReceivedEventEvent* is invoked.

4.6 Shared Memory Adapter project

The project is structured in a very similar way as its backend counterpart. It is implemented using the .NET's `MemoryMappedFile` class. Despite the name, it is the intended way to open the shared memory [19]. We do that by passing it the virtual file generated by the `boost::interprocess` library in the backend. Portions

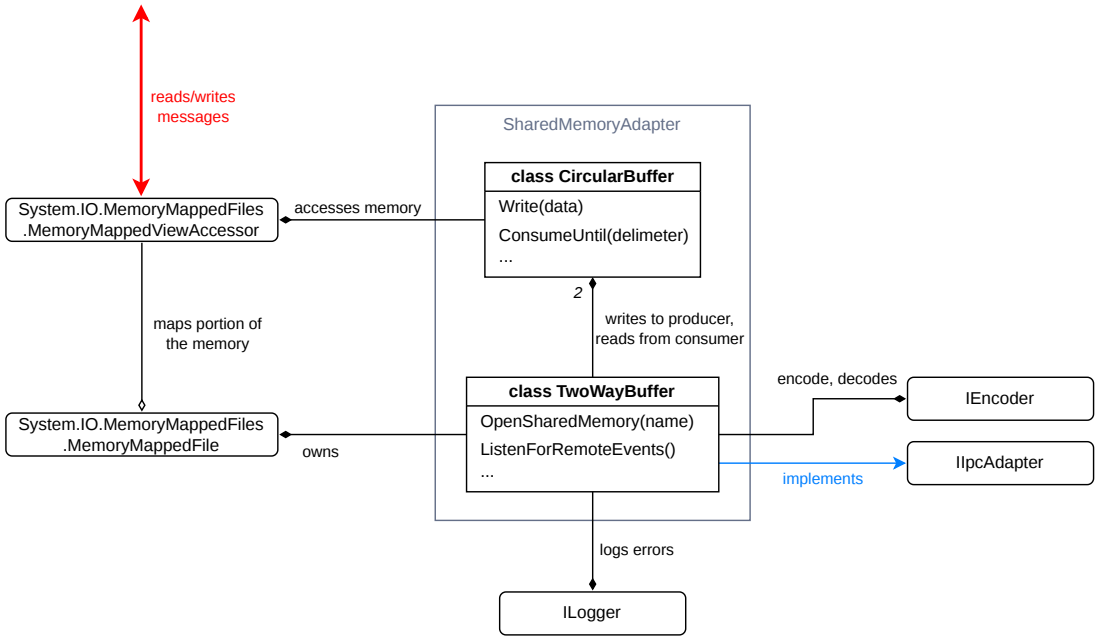

Figure 4.7 `SharedMemoryAdapter` project architecture

of the memory can be isolated and accessed via `MemoryMappedViewAccessor` which serves a similar purpose as the `mapped_region` type in `boost::interprocess`.

The `TwoWayBuffer` implementing the `IIpcAdapter` now plays the role of both the `TwoWayBuffer` and `shmemAdapter.cpp` in the backend. It opens the `MemoryMappedFile`, listens for messages, encodes events and passes them to its circular buffers. The `CircularBuffer` implementation is almost identical to the in the backend, with just a few caveats of the change in language. This time around, there is no `MemoryRegion` equivalent, as all the useful abstractions have already been built in to the `MemoryMappedViewAccessor`.

4.6.1 `CircularBuffer` class

The class owns the `MemoryMappedViewAccessor`. It still offers the `Write` and `ConsumeUntil` methods. We didn't define a dedicated iterator for the buffer so we need to implement finding the delimiter manually. The `Write` method is little less flexible, as it only accepts string and spans of bytes. In the backend, we converted any type into a span of bytes anyways, so the body of the method stays basically the same. Just instead of the `memcpy` function with pointers we use the accessor's `ReadArray` and `WriteArray` methods addressed directly with offsets. For full context see 3.4.2

4.6.2 `TwoWayBuffer` class

`TwoWayBuffer` is also very similar to how the same class and the adapter were implemented in the backend. It still opens and owns shared memory in the form of `MemoryMappedFile` object. It also owns and accesses both the consuming and the producing `CircularBuffer`. The `SendEvent` and `ListenForRemoteEvents` methods are equivalent to the `on_event` and `run_shmem_consumer` in `shmemAdapter.cpp` (see 3.4.4).

Only acquiring the shared memory object needs a little more work than before. Providing the object's name is not enough. We need to locate the virtual file created by the backend. This happens in the `OpenSharedMemory` method. The location changes drastically depending on the host's operating system. For specifics, see the subsection 5.3.3.

5 User documentation

5.1 Compiling and running the simulator with an Arduino sketch

5.1.1 Repository file structure

For this section it is useful to know, where are the important files located within the source code repository. For the full description of the repository directories, see A.1. These are some of the more noteworthy directories and files located in the root of the repository:

`backend/` contains source code of the backend simulation. The main `CMakeLists.txt` file is located here. It is also recommended to put the build directory here. The `CMakeLists` specifying the build targets and the `main()` function can be found within `backend/executable`.

`frontend/` contains source code of the frontend GUI and component management. The `Gui.sln` solution file is located here. The `Program.cs` with the `Main()` function is located within the `frontend/Main` directory. The build directory will be also located here, if not stated otherwise. The components definition directory is on the path `frontend/ComponentManagement/Components`. This path must be specified using `-c <path>` command line argument, if `frontend` is not the current working directory.

`tests/` contains example sketches and scenes used for testing both the backend and frontend.

`preprocess.sh`, `preprocess.bat` shell scripts performing basic sketch preprocessing using `arduino-cli`.

5.1.2 Build process and usage examples

First make sure you comply with the software prerequisites from the section 0.2. Make sure your Boost libraries can be found by the CMake metabuild system from your development environment. For further information, please follow the Boost's getting started guide [20]. After that, we can preprocess the sketch, compile it with the simulator, and run the simulator together with the frontend.

Backend process

1. Configure the CMake project in `backend/` with a `-DSKETCH` flag specifying the path to the sketch directory. This has to be done only when changing the sketch.

2. Build the configured CMake directory after any change to the sketch. Available targets are:

   `arguino` for IPC over shared memory (preferred).

arguino-tcp for IPC over TCP connection

3. The executable file will have the format `<build target>-<sketch name>`

For example, to run the `wave` sketch, you can execute these commands from the root of the repository: [1]

Listing 5.1 Example build process of the backend

```
cmake -S ./backend -B ./backend/build -DSKETCH=tests/wave
cmake --build ./backend/build --target arguino
./backend/build/executable/arguino-wave
```

Frontend process

1. Build `frontend/gui.sln`.
2. Run the `Gui` executable[2]with the desired scene. For example, executing from the `frontend` directory:

```
<build dir>/Gui -s ../tests/buttons.scene.yaml
```

   If you built the TCP version of the backend, be sure to add the flag `-i Tcp`.

For frontend and backend to communicate properly, it is necessary that they connect to the same IPC object. For TCP, it is the port (`-p` option for both programs), and for shared memory it is the shared memory's object name (`--shmem-name` option for both programs). These default to the same value for both of the aforementioned options, port `8888` and memory named `Arguino-ipc`.

GUI controls

Left mouse button Interact with controls (e.g. buttons)

Mouse wheel scroll Zoom and unzoom canvas

Mouse wheel press Pan canvas

Holding Alt key Hold mode. All controls clicked on will remain in a pressed state when clicked, until the control is clicked again. For example when holding multiple buttons is required.

5.1.3 Command-Line Arguments

Listed below are all the available command line options for the backend and frontend executables. Default values, if defined, are specified in the square brackets. Expected arguments are in the angle brackets. For arguments accepting only specific enumeration of values, all available values are listed in the angle brackets separated by a vertical bar (`|`). In square brackets is the default value used, when the option is omitted.

[1] The path of the actual executable may change depending on the build tool. The example shows usage with Linux and Clang. On Windows, the `.exe` extension is usually appended, i.e. `.../<target>-<sketch>.exe`.

[2] `Gui.exe` on Windows

Backend

`--log-simulator <path>` Path to the log file for simulator events. [`./core.log`]

`--log-ipc <path>` Path to the log file for Inter-Process Communication (IPC). [`./core_ipc.log`]

`-v, --verbosity <Debug | Info | Warning | Error>` Verbosity level of the log files. [`Info`]

Frontend

`-s, --scene <path>` Path to the `.yaml` scene definition file. [`./scene.yaml`]

`-c, --components <path>` Path to the directory containing component definitions. [`./ComponentManager/Components`]

`-i, --ipc <None | Tcp | Shmem>` Inter-process communication technology utilized for connecting with the simulator. [`Shmem`]

`--log-circuit <path>` Path to the log file for general circuitry events. [`./frontend.log`]

`--log-ipc <path>` Path to the log file for IPC events. [`./frontend_ipc.log`]

`-v, --verbosity <Debug | Info | Warning | Error>` Verbosity level of the log files. [`Info`]

Common for TCP usage

`-p, --port <int>` TCP port to listen to. [`8888`]

Common for shared memory usage

`--shmem-name <string>` Name of the memory-mapped file. [`Arguino-ipc`]

`--shmem-size <int>` Size of the memory-mapped region per circular buffer, specified in pages.[3] [`1`]

5.2 Defining components and scenes

Extensibility of the frontend through custom components and circuits is an important part the project. Here are necessary steps and requirements to create and use new components.

[3]On Linux and Windows, one page is usually 4096B.

5.2.1 Required file structure

All components must reside in one directory, referred to as *definitions directory*. Each component is represented by exactly one directory, referred to as *component directory*, placed in the definitions directory. The definitions directory must be supplied through the `-c` command line argument. These files must be present in the component directory:

- C# behavior script
- YAML component configuration
- One or more .svg images

The yaml file must have the same name as the component directory, i.e. `MyComponent/MyComponent.yaml`. The SVGs will be referenced in behavior script by their file name, excluding the .svg extension. For example `image.svg` will be referred to as `image`.

Notes on used YAML notation

In the next subsections, we will describe the expected structures of the YAML files using this notation:

- angle brackets must be replaced by proper parameters, i.e. `name: <string>`
- properties with question mark are optional, i.e. `optional?: ...`
- Vector2 is represented as a string of 2 float values separated by a space, i.e. `vector: 7.27 6.9`

5.2.2 Component configuration file specification

This file defines common properties of the custom component type.

Listing 5.2 YAML configuration file structure

```yaml
name: <string>
description?: <string>
imageSize?: <Vector2>
pins?: <uint>
pinNames?:
    - <string>
    - ...
    - ...
```

`name` Name of the component type as displayed in the UI. (not used yet)

`description` Brief description of the component as displayed in the UI. (not used yet)

`imageSize` Desired size of the component's sprites in pixels for default zoom level of the canvas. Defaults to <50, 50>.

pins Number of unnamed pins. These pins are identified by their 0-based index in both the scene definition and in the component logic.

pinNames List of unique names for each pin. These pins are identified by their given name in both the scene defintion and in the component logic.

Listing 5.3 Component configuration example: `Led.yaml`

```yaml
name: Led
description: Light emitting diode
imageSize: 50 50
pinNames:
  - anode
  - cathode
```

5.2.3 Component behavior script specification

How does the component interact with the rest of the circuit is described by the C# class with the named the same as the component type. This class must inherit from the abstract `Component` class. The file containing the class must be added to the `ComponentManagement.csproj` project. In order to compile, the user's class must also define a constructor like this:

```
    public MyComponent(string definitionPath)
    : base(definitionPath) { ... }
```

However, no code is required in the body of the constructor. The constructor can be used for initialization of properties known before the scene is created. The `Component` class provides most of the necessary API.

Component functions and properties

`virtual void OnInitialized()` Gets called when the component instance has been completed and all of its assets are available (config, pins, sprites, transform, etc.)

`virtual void OnPinConnected(Pin)` Gets called when a component's pin connects to an electric node. The connected pin is passed as argument.

`virtual void OnPinDisconnected(Pin)` Gets called when a component's pin disconnects from an electric node. The disconnected pin is passed as argument.

`virtual void OnPinStateChanged(Pin)` Gets called when a component's pin detected a change in digital state of a node it has been connected to. It can be thought of as changing input value on the pin. Pins set to write only mode by design don't detect this change. The detected pin is passed as argument.

`virtual void OnControlPress(Vector2)` Gets called when the component has been pressed on the circuit canvas. The cursor position at the time of the press is passed as argument.

`virtual void OnControlRelease` Gets called when the component has been released on the circuit canvas.

`void UpdateSprite(string)` Changes the .svg image drawn to the canvas. The new image is given by the svg name specified in the argument.

`Pin? GetPin(string)` Returns a Pin object with the name given in the argument. The name must be defined in the component's `namedPins` list. Returns null if no such pin exists.

`Pin? GetPin(uint)` Returns a Pin object with the ID given in the argument. The allowed range of Ids is from `0` to $pins + namedPins.Length$. With unnamed pins starting from the ID `0` and named pins starting from the ID `pins`. Returns null if the pin ID is out of range.

Pin functions and properties

`uint Pin.Id` Id of the pin, as described in the `GetPin(uint)` section.

`string Pin.Name` Name of the pin as described in the component configuration.

`bool Pin.IsDriving` True if the pin is driving. That is, if it actively forces the digital state on the connected node to be high. If the node has at least 1 driving pin, its digital state will be high. It will be low otherwise.

`bool Pin.IsHigh` True if the digital state of the connected node is high.

`bool Pin.IsLow` True if the digital state of the connected node is low.

`bool Pin.IsWriteOnly` Write only pins don't react to changes in the connected node's digital state.

`bool Pin.IsReadOnly` Read only pins cannot be set to driving.

`bool Pin.IsConnected` Whether is the pin connected to any electrical node.

`void Pin.MakeReadOnly()` Sets the pin to read only mode. Read only pins cannot be set to driving.

`void Pin.MakeWriteOnly()` Sets the pin to write only mode. Write only pins don't react to changes in the connected node's digital state.

`void Pin.MakeGeneral()` Sets the pin to general mode. General pins don't pose any restrictions to reading or writing.

`void Pin.SetValue(bool)` If the argument is true, it sets the pin to be driving. Otherwise, the pin set to not drive.

`void Pin.SetHigh()` Sets the pin to be driving. Same as `SetValue(true)`.

`void Pin.SetLow()` Sets the pin to be not driving. Same as `SetValue(false)`.

Listing 5.4 Button.cs

```
public class Button : Component {
    private const string SPRITE_BUTTON_UP =  "buttonUp";
    private const string SPRITE_BUTTON_DOWN =  "buttonDown";

    private bool _isPushed = false;

    public Button(string definitionPath) : base(definitionPath) {}

    internal override void OnInitialized() {
        UpdateSprite(SPRITE_BUTTON_UP);
    }

    public override void OnPinStateChanged(Pin pin)
        => UpdatePins();

    public override void OnControlPress(Vector2 cursorPosition) {
        _isPushed = true;
        UpdateSprite(SPRITE_BUTTON_DOWN);
        UpdatePins();
    }

    public override void OnControlRelease() {
        _isPushed = false;
        UpdateSprite(SPRITE_BUTTON_UP);
        UpdatePins();
    }

    private void UpdatePins() {
        UpdatePin(GetPin(0)!);
        UpdatePin(GetPin(1)!);
    }

    private void UpdatePin(Pin pin) {
        Pin otherPin = GetPin(1 - pin.Id)!;

        if (otherPin.IsHigh && _isPushed) {
            pin.SetHigh();
        }
        else {
            pin.SetLow();
        }
    }
}
```

In the example above, we have defined a behavior of a `Button` component. The component's configuration specifies two unnamed pins, and component's directory contains 2 SVG files, `buttonUp.svg` and `buttonDown.svg`. The pins are in the general mode, since we didn't specify their mode explicitly.

In the overridden `OnInitialized` we set the `buttonUp` image as the default sprite. In the overridden `OnControlPress` and `OnControlRelease` methods we update the internal state of the button: whether it is pressed or not. We change the actively displayed sprite accordingly.

`UpdatePins` is a helper method specific to this component. For both of the unnamed pins (retrieved by the `GetPin` method) it checks whether the value of the pin should be changed. A pin is set to be driving (`SetHigh` method), if the

button is pushed, and the other pin reads a high value. We have to be careful, just because a pin reads a high value, it doesn't mean we want to set it to driving. Once a state of a node drops back to low, we want states of both pins drop back to low as well, regardless of whether the button is pushed or not. [4]This update method is called every time a button is pressed, released, or a value on of its pins changes (`OnPinStateChanged` override).

5.2.4 Defining scenes

The scene YAML file describes component instances present in the scene, and how are their pins connected together in nodes. The path of this file is supplied through the `-s` command line argument. There are no restriction to where the scene file has to be placed.

Listing 5.5 YAML scene file structure

```
components:
  <Type name>:
    <Instance name>:
      position: <Vector2>
      rotation?: <float>
      scale?: <Vector2>
      extraProps?:
        <Property 1>: <Value 1>
        <Property 2>: <Value 2>
        ...
    <Instance name>:
      ...
nodes:
  - [ <Instance name>.<Pin name>, ...]
  - ...
  - ...
```

`components` Dictionary of component type objects. These objects are dictionaries of component instances of that type. These instances define properties of the individual components.

`Type name` String. Name of the component type. This must match the component directory name.

`Instance name` String. Name of the component instance. It identifies the instance within the node definition and in the UI. It must be unique across all instances within the components dictionary.

`position` Coordinates on the circuit canvas, with origin in the upper left corner. The coordinates must be positive, and lower than the expected size of the canvas in pixels. [5]

`rotation` Clockwise rotation angle in degrees on the circuit canvas.

`scale` Relative scaling along x and y axis on the circuit canvas.

[4]Unless, of course, the node at the other side is also high. That means the pin was, and will stay driving

extraProps A directory of custom properties defined in the component subclass as keys and values to assign to them. The properties must match the case and the type of the property. The property must be public and have a public setter.

nodes List of lists. The inner list represents all pins connected to the same electrical node. Its element is an instance-pin pair separated by a dot. Instance name must be present in the components dictionary. Pin name must be present in the component's configuration. In the case of unnamed pins, pin name is simply its index.

Listing 5.6 Scene example: `minimal.scene.yaml`

```yaml
Components:
  Led:
    Led1:
      Position: 100 100
    Led2:
      Position: 200 100
  Button:
    Btn:
      Position: 100 200
  Vcc:
    Source:
      Position: 125 300
Nodes:
  - [Source.0, Btn.0]
  - [Led1.cathode, Btn.1]
  - [Led2.cathode, Btn.1]
```

The example scene in the listing 5.6 contains 2 LEDs, 1 button and 1 source of voltage. Source, which has one pin that is always driving, is connected to the 0th pin of the button, which reads a high value of the node. Both LEDs are connected to the 1st pin of the same button. Pressing the button will copy the digital state from pin 0 to pin 1, pin 1 becomes driving, and thus lights up the LEDs.

5.3 IPC interface

This section describes how to set up IPC communication of a custom frontend application with the backend simulator. The two processes can communicate together either via TCP connection or through shared memory. The only requirement for a TCP connection, is that the client on the frontend listens to the same port as the server on the backend. IPC over shared memory requires a few more considerations, so that the memory accesses match the memory layout of shared internal structure created by the backend. This structure is described in the subsection 5.3.3;

The processes communicate using concrete events with up to 3 integer arguments. These events are encoded into a human-readable text messages, separated by a common delimiter. The messages are then sent over IPC.

[5]This is a technical limitation of one of the Avalonia libraries used. See 4.3.5

5.3.1 Text message format

Individual messages are delimited by the semicolon character (`;`). The individual values within one message are delimited by the colon character (`:`). The integer values are written in decimal system, using the ASCII characters of the actual digits. The numbers can be padded with zeros.[6]The semantics of these values are better described in the chapter about architecture, subsection 2.2.3. The format of the message then looks as the following:

Listing 5.7 Format of an IPC text message

```
<sim_time>:<lvt>:<event_type>:<arg1>:<arg2>:<arg3>;
```

`sim_time` Simulation time, 64-bit signed integer. Real time elapsed from the start of the backend's simulation represented in microseconds. The backend has no use for this value.

`lvt` Local virtual time, 64-bit signed integer. It describes an order in which the event occurred, relative to other events.

`event_type` A single character representing the type of the event. This type decides the action taken upon consuming the event. Recognized event types and their corresponding characters are described in the next subsection.

`arg1, arg2, arg3` 32-bit signed integers. Values have meaning depending on the event. Arguments not defined by the event type can be omitted. Make sure that the number of colons match the number of passed arguments.

For example, this next message encodes the `Write` event passing 2 arguments at the simulation time of 264 seconds:

Listing 5.8 Example of an IPC text message

```
000264997488:0000016:W:08:1;
```

5.3.2 Recognized event types

Write event

Signalizes digital value being written into a pin. If consumed by the backend, the value gets written to the internal `ArduinoState`. If consumed by the frontend, it should reflect the changes caused by the change of the pin's state. This event is produced by the backend on `digitalWrite` function call within the sketch.

Encoding character: `W`

Arguments:

1. **Pin index** of the digital pin from the range 0–13.
2. **Value** 0 for the low value, 1 for the high value.

[6]The actual implementations pad the numbers in such a manner, that all messages of the same event type have the same length.

Pinmode event

Signalizes change of a pin mode of a pin. If consumed by the backend the value gets written to the internal `Arduinostate`. If consumed by the frontend, it should change the behavior of the pin, such that only input pins are able to send the `Write` event. This event is produced by the backend on `pinMode` function call within the sketch.

Encoding character: `P`

Arguments:

1. **Pin index** of the digital pin from the range 0–13.
2. **Mode** 0 for the output mode, 1 for the input mode.

Reboot event

Produced by the backend, when the simulation has started over. Useful, when the backend process has be terminated and then executed again. At this point the frontend has to invalidate its current state and start again. Backend itself does no action upon consuming this event (it may force the reboot of the simulation in the future).

Encoding character: `R`

Arguments: None

5.3.3 Shared memory specification

Locating memory mapped file

On Linux, the mapped memory region is orchestrated by a proper shared memory object. It can be accessed either by a syscall, or by reading the `\dev\shm\<shmem_name>` virtual file.

On Windows, the shared memory is emulated by the `Boost.interprocess` library via a memory mapped file. It can be located at the path specified by the `Common AppData` registry. For Windows Vista and newer, this path defaults to `C:\ProgramData`. On this path, a `boost_interprocess` directory is created. The memory mapped file, named the same as the shared memory object, can be found in one if its subdirectories. [21]

In either case, the name of the read memory mapped file or shared memory object has to match the name given to the mapped region via the `--shmem-name` flag of the backend command.

Memory layout

The mapped memory region is split into 2 equal sections, both containg the same circular buffer data structure. So for a mapped region of size n bytes, and for the start address of the region s_a, the second buffer is located on the memory address $s_a + n/2$ (n is always even, as it is a multiple of a page size). Let's denote the start address of a buffer s_b

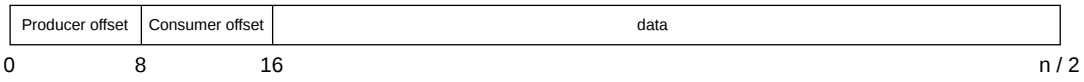

Figure 5.1 Memory layout of the circular buffer

The buffer itself has 2 control variables stored within it. Both are 64-bit unsigned integers representing a byte offset from the beginning of the buffer's data. The first, located at s_b, is the offset at which the producer will write the next chunk of data into the buffer. The second, located at $s_b + 8$, is the offset at which the next chunk of data will be read from the buffer, if present. From the address $s_b + 16$ until the next buffer, or the end of the mapped region, the actual data are stored. That means the actual capacity of data stored within one buffer is $n/2 - 16$.

The data have no uniform size, structure or address alignment. The length of a message in the buffer must be deduced from the data within the message itself. In case of the text encoding, it is the message delimiter. A message can be located at the boundary of the data region. In that case the message is split so that every byte until the very end of the region is filled, and the rest is written at $s_b + 16$ again. There are no gaps between the messages.

The frontend process is responsible for consuming messages from the first buffer and producing them to the second one. As such, the consumer offset of the first buffer and the producer offset of the second buffer must be updated accordingly. The other two offsets must remain read only. The backend process has these roles reversed.

Conclusion

In the end we managed to develop two nontrivial applications. One for running and simulating the user's Arduino code. The other one for creating a interactive virtual environment, where the Arduino can be connected into a circuit with visual feedback. In chapter 2 of this thesis we described the thought process, project structure, and technologies used in order to achieve the goals we set for ourselves in section 0.2. Then we looked into more detail on how we implemented these technologies while avoiding many pitfalls in chapters 3 and 4.

While we implemented only a couple of most used functions from the Arduino library, and the user interface leaves a lot to be desired, we believe the current version of the product represents a solid foundation on which new functions, events, components and quality of life features can be laid down. One of the biggest hurdles was making sure that the two applications stay synchronized. We succeeded in that regard, by learning a lot of useful information about interprocess communication and approaches commonly used in distributed computing.

The user can extend their component library just by following the specification in section 5.2. They can use custom vector graphics, and give life to their components thanks to the abstract `Component` class. Then they can add it into a scene together with Arduino running their own code. All this while all the heavy work is done by the reflection and parsing machinery on the background.

The next step would be trying the system out on more complicated components, like entire shields with shift registers and 7-segment displays. Further flexibility could be added by composing components into bigger components, in a way similar to how scenes are created. GUI could be improved by many useful features, like editing scenes and components, displaying wires, manipulating the simulation and much more.

Bibliography

1. *Sketch specification | Arduino Documentation* [online]. [visited on 2026-06-25]. Available from: https://docs.arduino.cc/arduino-cli/sketch-specification/.
2. *UNO R3 SMD | Arduino Documentation* [online]. [visited on 2026-06-25]. Available from: https://docs.arduino.cc/hardware/uno-rev3-smd/.
3. *digitalRead() | Arduino Documentation* [online]. [visited on 2026-06-25]. Available from: https://docs.arduino.cc/language-reference/en/functions/digital-io/digitalread/.
4. *Sketch build process | Arduino Documentation* [online]. [visited on 2026-06-25]. Available from: https://docs.arduino.cc/arduino-cli/sketch-build-process/.
5. FUJIMOTO, R.M. Parallel and distributed simulation systems. In: *Proceeding of the 2001 Winter Simulation Conference (Cat. No.01CH37304)* [online]. Arlington, VA, USA: IEEE, 2001, pp. 147–157 [visited on 2026-03-05]. ISBN 978-0-7803-7307-5. Available from DOI: 10.1109/WSC.2001.977259.
6. *Choosing a custom control type | Avalonia Docs* [online]. 2026-04-20. [visited on 2026-05-19]. Available from: https://docs.avaloniaui.net/docs/custom-controls/choosing-a-custom-control-type.
7. *The MVVM pattern | Avalonia Docs* [online]. 2026-04-20. [visited on 2026-05-19]. Available from: https://docs.avaloniaui.net/docs/fundamentals/the-mvvm-pattern.
8. *IEEE Xplore Full-Text PDF:* [online]. [visited on 2026-07-07]. Available from: https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=257627.
9. *Daytime.3 - An asynchronous TCP daytime server* [online]. [visited on 2026-07-12]. Available from: https://www.boost.org/doc/libs/latest/doc/html/boost_asio/tutorial/tutdaytime3.html.
10. CPPCON. *CppCon 2016: Michael Caisse "Asynchronous IO with Boost.Asio"* [online]. 2016. [visited on 2026-07-12]. Available from: https://www.youtube.com/watch?v=rwOv_tw2eA4.
11. *AvaloniaUI/Avalonia.NameGenerator* [online]. Avalonia UI, 2026 [visited on 2026-07-11]. Available from: https://github.com/AvaloniaUI/Avalonia.NameGenerator. original-date: 2020-10-25T17:51:18Z.
12. *Data context | Avalonia Docs* [online]. 2026-04-20. [visited on 2026-07-11]. Available from: https://docs.avaloniaui.net/docs/data-binding/data-context.
13. *How to use INotifyPropertyChanged | Avalonia Docs* [online]. 2026-04-02. [visited on 2026-07-11]. Available from: https://docs.avaloniaui.net/docs/data-binding/inotifypropertychanged.
14. *Avalonia property system | Avalonia Docs* [online]. 2026-04-20. [visited on 2026-07-11]. Available from: https://docs.avaloniaui.net/docs/properties/.

15. *Skia* [Skia] [online]. [visited on 2026-07-11]. Available from: https://skia.org/.

16. DOTNET-BOT. *SKCanvas.Save Method (SkiaSharp)* [online]. [visited on 2026-07-11]. Available from: https://learn.microsoft.com/en-us/dotnet/api/skiasharp.skcanvas.save?view=skiasharp.

17. *Avalonia samples - SnowflakesControlSample* [GitHub] [online]. [visited on 2026-07-11]. Available from: https://github.com/AvaloniaUI/Avalonia.Samples/blob/08b578f80f5d1b4b9eb52f5c41b6128621437240/src/Avalonia.Samples/CustomControls/SnowflakesControlSample/Controls/ScoreRenderer.cs.

18. KARELZ. *NetworkStream.ReadAsync Method (System.Net.Sockets)* [online]. [visited on 2026-07-12]. Available from: https://learn.microsoft.com/en-us/dotnet/api/system.net.sockets.networkstream.readasync?view=net-10.0.

19. ADEGEO. *Memory-Mapped Files - .NET* [online]. [visited on 2026-07-12]. Available from: https://learn.microsoft.com/en-us/dotnet/standard/io/memory-mapped-files.

20. *Boost User Guide - Getting started* [online]. [visited on 2026-06-27]. Available from: https://www.boost.org/doc/user-guide/getting-started.html.

21. *Sharing memory between processes* [online]. [visited on 2026-06-29]. Available from: https://www.boost.org/doc/libs/latest/doc/html/interprocess/sharedmemorybetweenprocesses.html.

List of Figures

Listings

A Attachments

A.1 Source code repository

The file attachment `thesis.zip` contains the repository with the source code in its entirety. The repository is equivalent with the public git repository found at the URL `https://github.com/TenIdiotZInternetu/Arguino` at the tag add tag.

The file structure of the repository is as follows:

`.vscode/` contains the files:

- **`launch.json`** specifies backend debug configurations using the `cppdbg` VS Code extension.
- **`settings.json`** specifies backend source and build directories using the `CMakeTools` VS Code extension.

`backend/` contains the source code for the backend application. Each directory corresponds to a single module described in the chapter 3, with the exception of two:

- **`libs/`** contains the two utilities modules: `logger` and `timer`.
- **`.vscode/`** contains the setup equivalent to the `.vscode` directory in the root of the repository.

Other files in this directory are:

- **`.clang-format`** specification of the automatic code formatting.
- **`.gitignore`** exclusion of build files.
- **`CMakeLists.txt`** main metabuild specification for the backend. References CMake files in the module subdirectories.
- **`CMakePresets.json`** default build configurations using Clang on Linux and MSVC on Windows.
- **`vcpkg.json`** specifies Vcpkg dependencies for the Boost libraries.
- **`preprocess.sh`** shell script performing basic sketch preprocessing using `arduino-cli`. For use on Linux.
- **`preprocess.bat`** same as `preprocess.sh` but for the use on Windows.

`docs/` Contains year project specification and other documentation:

- **`RP_Specifikacia.odt`, `RP_Specifikacia.pdf`** final year project specification in Slovak language.
- **`specification-draft.md`** rough version of the year project specification in English language. Contains the original vision and expectations for the project.
- **`component_addons_doc.md`** contains specification of custom components and scenes configuration in the scope of the section 5.2.

`frontend/` contains the source code for the frontend application. Each directory corresponds to a single project described in the chapter 4. The remaining files are `.gitignore` and the `gui.sln` .NET solution file.

`res/` contains assets used in source code unrelated context, e.g. screenshots.

`tests/` contains example sketches and scenes used for testing both the frontend and the backend.

`thesis/` contains source files for this document, e.g. `.tex` files, bibliography files and figures.

`.gitignore` excludes build and Visual Studio files

`README.md` describes the usage and build process of both the backend and the frontend in the scope of the section 5.1.